

PHOTON-NATIVE QUANTUM PERCEPTRONS AND NEURAL NETWORKS

by **Md Shadnan Azwad Khan**

Supervised by **Dr. Daphné Wang**

as part of a research internship at the
Algorithms & Applications Team
QUANDELA

A thesis submitted in partial fulfillment for the degree of
Master Informatique – Quantum Information

in the
Faculty of Science & Engineering
SORBONNE UNIVERSITÉ

March 3, 2025 – September 3, 2025

“We demand rigidly defined areas of doubt
and uncertainty!”

— **Douglas Adams**

The Hitchhiker's Guide to the Galaxy

“... by suitably coupling the influence with
the result ... you can make the machine
almost simulate a conscious behaviour...”

— **Satyendra Nath Bose**

Man in the Scientific Age

ABSTRACT

Photonic quantum neural networks promise fast, low-energy learning but must reconcile linear optics with nonlinear activation. This thesis advances a spiking-inspired, photon-native approach in the discrete-variable regime: reconfigurable interferometers perform learned linear transformations, and photon detection with classical feed-forward supplies effective nonlinearity. We first realize a photon-native quantum perceptron that treats photon occupations as spike events and can express decision boundaries beyond a classical perceptron. We then build layers and networks that measure outputs per shot and pass compact classical signals between units, enabling stable training without long quantum memories. Across standard tasks, these photon-native networks learn reliably and remain competitive with multilayer perceptrons at comparable scale.

Keywords: Photon-native Quantum Computing, Quantum Neural Networks, Quantum Perceptrons, Machine Learning, Neuromorphic Computing.

ACKNOWLEDGEMENTS

I am deeply grateful to my supervisor, Dr. Daphne Wang, for encouraging me to explore a range of ideas and for sharing my enthusiasm for neuromorphic computing. I also thank the Algorithms and Applications team for the generous discussions and insights that shaped this work.

CONTENTS

Abstract	ii
Acknowledgements	iii
List of Figures	v
1 Introduction	1
2 Photon-native Quantum Computation	2
2.1 Bosonic modes and Fock space	2
2.2 Passive linear optics	2
2.3 Interferometer architectures and unitary decompositions	3
2.4 Many-photon interference	3
2.5 Photodetection and measurement	4
2.6 Feed-forward operations	5
2.7 Photon-native algorithms	5
3 What is a Neuron?	6
3.1 Artificial Neurons: the Perceptron	6
3.2 Spiking neurons and the Spike-Response Model	6
3.3 ANN-SNN correspondence	8
4 From Perceptrons to Neural Networks	10
4.1 Feedforward Neural Composition	10
4.2 Expressivity and Generalization	11
4.3 Learning Objectives and Optimization	11
4.4 Quantum Neural Networks	11
5 Quantum Perceptrons	13
5.1 Quantum Perceptron Model	13
5.2 Nonlinearity and activations	15
5.3 Function and properties	17
6 Photon-native Quantum Perceptron	18
6.1 Data Encoding as Unitary Feature Maps	18
6.2 Unitary-Parametrized Affine Map	18
6.3 Measurement-Conditioned Quantum Activation	19
6.4 A Minimal Architecture	22
6.5 Simulations and Training	23
7 Photon-native Quantum Neural Network	28
7.1 Photon-native perceptron layer	29
7.2 Photon-native QNN	29
7.3 Image Classification	30
8 Conclusion	37
References	38

LIST OF FIGURES

2.1	Universal N -mode interferometer meshes	3
2.2	Boson sampling setup	4
2.3	Near-to mid-term photonic quantum computing setup	5
3.1	From realistic neuronal models to abstractions	9
4.1	Multilayer perceptron (MLP)	10
4.2	Parameterized quantum circuit (PQC)	12
6.1	Mean accuracy comparison for the Quantum Perceptron and baseline classical models	24
6.2	Results for the XOR dataset.	25
6.3	Decision boundaries for the Circles dataset under varying ring factor and label noise.	26
6.4	Activation surfaces for the Moons dataset as label noise increases.	27
7.1	Photon-native quantum neural network with classical communication.	28
7.2	Visualization of the reduced 64-D features	31
7.3	Reduced MNIST Benchmark (PnQNN: 450k; MLP: 4.7k).	33
7.4	Reduced MNIST Benchmark (PnQNN: 118k; MLP: 152k).	34
7.5	Reduced CIFAR-10 Benchmark (PnQNN: 450k; MLP: 4.7k).	35
7.6	Reduced CIFAR-10 Benchmark (PnQNN: 118k; MLP: 152k).	36

INTRODUCTION

Modern machine learning delivers remarkable results, but often at high computational and energy cost. This motivates architectures that align with the physics of the device and are lightweight and energy-efficient. Neuromorphic computing offers one such direction. Rather than continuous, always-on activations, spiking systems process discrete *events*: inputs integrate until a threshold is crossed, a spike is emitted, and the state resets. Event-driven operation yields sparsity and energy use proportional to activity [1].

Quantum computing explores models of computation that exploit superposition, interference, and measurement to tackle tasks believed to be classically intractable. While fault-tolerant gate-based machines remain a long-term goal, near-term work increasingly emphasizes algorithms and architectures compatible with today’s hardware [2].

Discrete-variable (DV) photonic processors manipulate bosonic modes using passive linear optics and read out with photodetectors. Universal interferometer meshes enable fast, energy-efficient linear mixing in Fock space, with configuration and detection dominating the resource cost [3, 4, 5]. Measurement produces samples; repeated shots yield detection statistics, from which we can estimate expectation values. What DV photonics lacks are strong, scalable optical nonlinearities; measurement with classically controlled feed-forward offers a practical route to effective nonlinearity [6].

* * *

Aim. In this thesis we explore a *photon-native* approach to neural computation in the DV regime. The core intuition is simple: linear optics mix signals, and measurement supplies the nonlinearity. Interpreting spatial photon occupation as spike events, we will assemble perceptrons, layers, and small networks from standard DV components. The emphasis will be on clear abstractions that link neuromorphic intuition to discrete-variable photonic quantum computation.

Organization. First, we introduce photon-native quantum-computing primitives in [chapter 2](#). We then discuss neuronal abstractions (perceptrons and spiking neurons) that motivate this study in [chapter 3](#). Feed-forward composition in classical neural networks is formalized in [chapter 4](#), which also provides a brief survey of quantum neural networks for context. We review the quantum perceptron literature through a quantum information lens in [chapter 5](#). A proposed photon-native quantum perceptron algorithm is presented and evaluated on small classification tasks in [chapter 6](#). These units are assembled into layers and networks in [chapter 7](#), including a proof-of-concept comparison with multilayer perceptrons. Finally, we conclude with a brief outlook in [chapter 8](#).

PHOTON-NATIVE QUANTUM COMPUTATION

This chapter sets up a DV photonic framework: Fock states, passive linear optics, and classically controlled adaptive feed-forward. We use the term photon-native following Salavrakos et al. [7]. The framework we describe is motivated by Quandela's full stack, its photonic quantum processing units, classical simulators, and ongoing laboratory efforts aimed at near-term realization.

2.1 | BOSONIC MODES AND FOCK SPACE

We consider m spatial modes with annihilation/creation operators $\{a_j, a_j^\dagger\}$ acting on the full Fock space. They obey

$$[a_j, a_k^\dagger] = \delta_{jk}, \quad [a_j, a_k] = [a_j^\dagger, a_k^\dagger] = 0, \quad a_j|0\rangle = 0, \quad (2.1)$$

where $|0\rangle$ is the vacuum and the identity. The number operator in mode j is

$$\hat{n}_j = a_j^\dagger a_j, \quad \text{with eigenvalues } n_j \in \mathbb{N}_0, \quad \hat{N} = \sum_{j=1}^m \hat{n}_j, \quad (2.2)$$

implying $[\hat{n}_j, a_k] = -\delta_{jk}a_k$, $[\hat{n}_j, a_k^\dagger] = \delta_{jk}a_k^\dagger$. A multimode Fock state $|\mathbf{n}\rangle \equiv |n_1, \dots, n_m\rangle$ (with occupations n_j) is

$$|\mathbf{n}\rangle = \prod_{j=1}^m \frac{(a_j^\dagger)^{n_j}}{\sqrt{n_j!}} |0\rangle, \quad \langle \mathbf{n} | \mathbf{n}' \rangle = \delta_{\mathbf{n}, \mathbf{n}'}. \quad (2.3)$$

The N -photon sector collects all states with total occupation $\sum_j n_j$:

$$\mathcal{H}^{(N)} = \text{span}\{|\mathbf{n}\rangle : \sum_j n_j = N\}, \quad \dim \mathcal{H}^{(N)} = \binom{N+m-1}{N}. \quad (2.4)$$

We denote input and output occupation vectors by $\mathbf{n}^{\text{in}}$ and $\mathbf{n}^{\text{out}}$. A single-photon source in mode j prepares $|1_j\rangle = a_j^\dagger|0\rangle$; arrays realize prescribed occupations in $\mathcal{H}^{(N)}$.

2.2 | PASSIVE LINEAR OPTICS

An ideal, number-preserving interferometer effects a Heisenberg-picture linear transformation of the field operators. With $\mathbf{a} = (a_1, \dots, a_m)^\top$,

$$\hat{U}^\dagger \mathbf{a} \hat{U} = U \mathbf{a}, \quad \hat{U}^\dagger \mathbf{a}^\dagger \hat{U} = \mathbf{a}^\dagger U^\dagger, \quad (2.5)$$

where $U \in U(m)$ is the (unitary) scattering matrix. Off-diagonal U_{ij} indicate coupling between modes i and j .

The core interferometric components are:

- **Phase shifter** on mode j :

$$P_j(\phi) = e^{i\phi\hat{n}_j} \quad a_j \mapsto e^{i\phi} a_j. \quad (2.6)$$

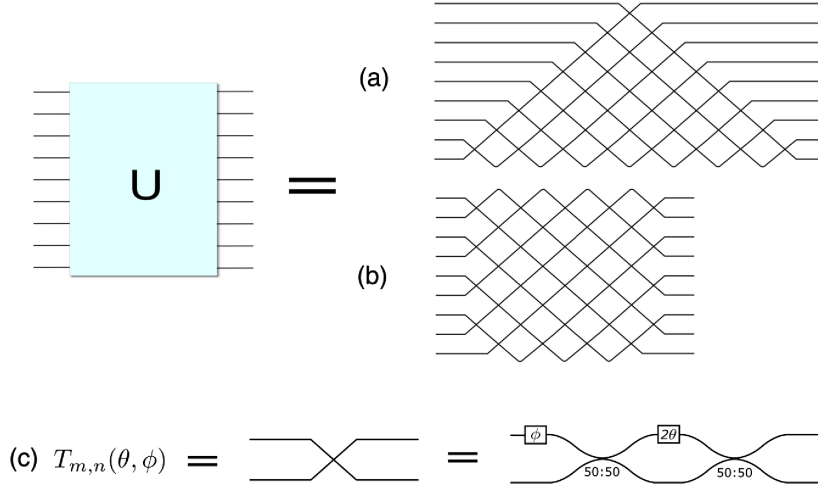


Figure 2.1: Universal N -mode interferometer meshes, (a) Reck (triangular) and (b) Clements (rectangular). Crossings are variable beam splitters $T_{m,n}(\theta, \phi)$ (implemented via MZIs) [9].

- **Beam splitter** on modes (p, q) , acting on $(a_p, a_q)^\top$:

$$B(\theta, \varphi) = \begin{pmatrix} \cos \theta & e^{i\varphi} \sin \theta \\ -e^{-i\varphi} \sin \theta & \cos \theta \end{pmatrix}. \quad (2.7)$$

with *transmissivity* $T = \cos^2 \theta$ and *reflectivity* $R = \sin^2 \theta$ (so $T + R = 1$); φ sets the relative phase.

- **Mach-Zehnder interferometer** implements any $U(2)$ on (p, q) :

$$U_{(p,q)}(\alpha, \theta, \beta) = \begin{pmatrix} e^{i\alpha} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & e^{i\beta} \end{pmatrix}. \quad (2.8)$$

Conventions may shift phases between elements; the physical content is unchanged.

2.3 | INTERFEROMETER ARCHITECTURES AND UNITARY DECOMPOSITIONS

Any $U \in U(m)$ can be synthesized from nearest-neighbor $U(2)$ rotations (MZIs) and single-mode phase shifts. Two standard meshes (Figure 2.1) are:

- **Reck (triangular) mesh** [8]: the unitary-triangular (QR) factorization obtained via complex Givens rotations that zero subdiagonal entries column-by-column. It uses $m(m-1)/2$ two-mode blocks and has *optical depth* $2m-3$, where depth is the maximum number of two-mode couplers along any input-output path.
- **Clements (rectangular) mesh** [9]: a rearrangement of the same number, $m(m-1)/2$, of nearest-neighbor $U(2)$ (two-mode) blocks into alternating even/odd layers, yielding balanced path lengths (every path crosses the same number of couplers and phase shifters) and depth m .

Both meshes realize m^2 real degrees of freedom: $m(m-1)/2$ coupling angles (from the two-mode blocks) plus $m(m+1)/2$ phases. Factoring out trivial input and output rephasings ($2m$ phases, with one overall global redundancy) leaves $(m-1)^2$ internal real degrees of freedom for the mesh.

2.4 | MANY-PHOTON INTERFERENCE

Let $U \in U(m)$ act on modes as $a'_i = \sum_j U_{ij} a_j$. The induced evolution on the N -photon sector $\mathcal{H}^{(N)}$ is the bosonic (symmetric) representation $U^{(N)}$. For an input occupation $\mathbf{n}^{\text{in}}$ and output occupation $\mathbf{n}^{\text{out}}$ (both

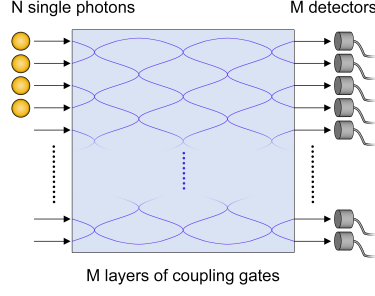


Figure 2.2: Boson sampling setup: N single photons are injected into an M -mode linear interferometer implementing a Haar-random unitary $U \in U(M)$, and photon-counting detectors measure the resulting output occupation pattern [10].

summing to N), the standard scattering amplitude is

$$\langle \mathbf{n}^{\text{out}} | U^{(N)} | \mathbf{n}^{\text{in}} \rangle = \frac{\text{Per}(U_{\mathbf{n}^{\text{out}}, \mathbf{n}^{\text{in}}})}{\sqrt{\prod_{i=1}^m n_i^{\text{out}}! \prod_{j=1}^m n_j^{\text{in}}!}}, \quad \Pr(\mathbf{n}^{\text{out}} | \mathbf{n}^{\text{in}}) = |\langle \mathbf{n}^{\text{out}} | U^{(N)} | \mathbf{n}^{\text{in}} \rangle|^2. \quad (2.9)$$

Here $U_{\mathbf{n}^{\text{out}}, \mathbf{n}^{\text{in}}}$ is the $N \times N$ matrix obtained from U by repeating column j exactly n_j^{in} times and row i exactly n_i^{out} times. The permanent reflects coherent addition over all bijections between indistinguishable bosons and underpins the classical hardness of boson sampling. Setup shown in Figure 2.2.

Some standard cases include:

- *Collision-free, distinct modes* ($n_j^{\text{in}}, n_i^{\text{out}} \in \{0, 1\}$): if I_{in} and I_{out} are the occupied input and output index sets,

$$\Pr(\mathbf{n}^{\text{out}} | \mathbf{n}^{\text{in}}) = |\text{Per}(U_{I_{\text{out}}, I_{\text{in}}})|^2. \quad (2.10)$$

- *Two photons*: for inputs $j_1 \neq j_2$ and outputs $i_1 \neq i_2$,

$$P(i_1, i_2 | j_1, j_2) = \frac{1}{2} |U_{i_1 j_1} U_{i_2 j_2} + U_{i_1 j_2} U_{i_2 j_1}|^2, \quad (2.11)$$

with bunching obtained by repeating rows/columns accordingly (Hong–Ou–Mandel at a 50:50 beam splitter).

For any input Fock state $|\mathbf{n}^{\text{in}}\rangle$, the mean photon number in output mode i is

$$\langle \hat{n}_i^{\text{out}} \rangle = \sum_{j=1}^m |U_{ij}|^2 n_j^{\text{in}}, \quad \langle \mathbf{n}^{\text{out}} \rangle = |U|^{\circ 2} \mathbf{n}^{\text{in}}. \quad (2.12)$$

Thus single-photon propagation reduce to a classical stochastic map, while the *full* $N \geq 2$ distribution retains permanent-level complexity [11].

2.5 | PHOTODETECTION AND MEASUREMENT

Measurements act mode-locally; a joint measurement is the tensor product over the measured modes.

Photon-number resolving (PNR). An ideal PNR detector on mode i has POVM elements $M_i^{(k)} = |k\rangle\langle k|_i$ for $k = 0, 1, 2, \dots$. Measuring all modes returns $\mathbf{n}^{\text{out}}$ distributed as in §2.4.

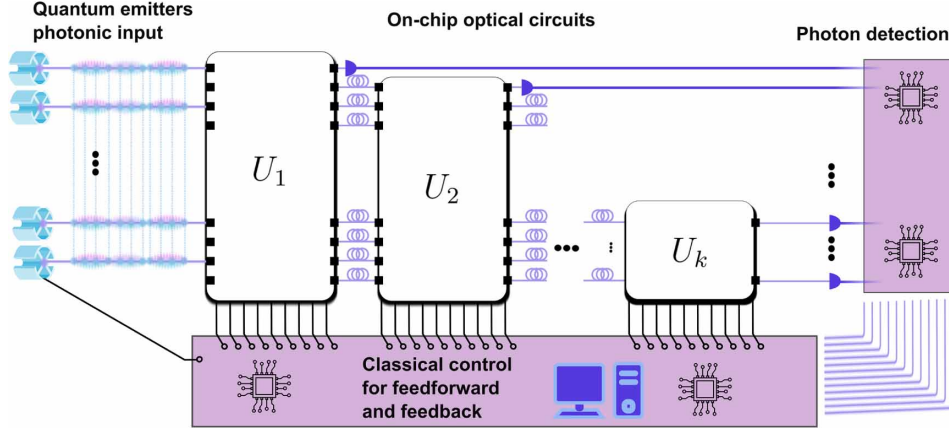


Figure 2.3: Near-to mid-term photonic quantum computing setup: synchronized single photons enter a photonic integrated circuit implementing a unitary U_1 with direct photon detection, while advanced architectures add feedforward and delays so subsequent blocks $U_2, \dots, U_k$ depend on intermediate measurements, and can introduce nonlinearity by supplying entangled resource states from light–matter interactions [7].

Threshold detection. Ideal threshold detection distinguishes vacuum from “ ≥ 1 ”:

$$M_i^{(\text{no})} = |0\rangle\langle 0|_i, \quad (2.13)$$

$$M_i^{(\text{click})} = \mathbb{I}_i - |0\rangle\langle 0|_i. \quad (2.14)$$

Click-pattern probabilities are coarse-grainings of the PNR distribution.

2.6 | FEED-FORWARD OPERATIONS

A single adaptive step measures some modes (outcome y with measurement represented by a quantum instrument $\{\mathcal{R}_y\}$) and applies a reconfigured interferometer $U(\lambda_y)$ on the remaining modes:

$$\Phi(\rho) = \sum_y U(\lambda_y) \mathcal{R}_y(\rho) U(\lambda_y)^\dagger. \quad (2.15)$$

Composing such steps yields a decision tree whose branches are the distinct measurement outcomes with classical control selecting the next unitary. This is the standard mechanism used broadly in photon-native protocols to implement conditional transformations and effective nonlinear behavior. Layout in Figure 2.3.

2.7 | PHOTON-NATIVE ALGORITHMS

In our framework *photon-native* refers to protocols that *directly* use single-photon Fock-state preparation, linear interferometers, photon counting, and (optionally) adaptivity, without relying on deterministic two-qubit entangling gates. Typical classes include:

- **Sampling and combinatorial tasks:** fixed-depth interferometers with multi-photon inputs whose output statistics encode permanents; structure in U (e.g., from graphs) can be probed via detection statistics [12].
- **Parametric interferometers for learning:** programmable meshes $U(\theta)$ with PNR readout define models $x \mapsto \mathbb{E}_\theta[\text{counts}]$ for discriminative tasks, generative modeling, or kernels; expressivity stems from interference and measurement. [13, 14, 15]
- **Adaptive linear-optical protocols:** mid-circuit measurements and feed-forward (and, in some settings, state injection) extend what is reachable with passive optics alone and are central to measurement-based and near-term photonic schemes [16, 17].

WHAT IS A NEURON?

This report does not explore the detailed mechanics of neuronal dynamics in mammalian or other animal brains. That topic lies outside the scope of this work, which focuses on neuromorphic computing (among other topics) rather than on computational neuroscience. At any rate, the author is thoroughly unqualified to address those matters. For a comprehensive treatment, consult the excellent resource in [18].

3.1 | ARTIFICIAL NEURONS: THE PERCEPTRON

A more appropriate question, then, is: What is an artificial neuron?

The artificial neuron, or perceptron, implements a parametric nonlinear mapping

$$f : \mathbb{R}^d \rightarrow \mathbb{R}, \quad f(\mathbf{x}) = \phi(\mathbf{w}^\top \mathbf{x} + b), \quad (3.1)$$

where $\mathbf{x} \in \mathbb{R}^d$ is the input vector, $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $b \in \mathbb{R}$ the bias, and $\phi : \mathbb{R} \rightarrow \mathbb{R}$ a nonlinear activation (ReLU, sigmoid, tanh, etc.) [19, 20, 21, 22]. By choosing ϕ one fixes the function class

$$\mathcal{F}_\phi = \{ f_{\mathbf{w},b} : \mathbf{x} \mapsto \phi(\mathbf{w}^\top \mathbf{x} + b) \mid \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R} \}.$$

The analytic properties of ϕ , such as continuity, differentiability, boundedness, Lipschitz constant, homogeneity and monotonicity, directly determine:

- Expressive power of $\mathcal{F}_\phi$, since universal approximation requires sufficiently expressive ϕ [23, 24];
- Gradient dynamics, as the form and smoothness of ϕ' and higher derivatives govern vanishing or exploding gradients [25];
- Inductive bias, because ϕ 's range, saturation, and scaling impose constraints on the functions a network can learn [26, 27].

In modern machine learning, the perceptron tuple $(\mathbf{w}, b, \phi)$ serves as the fundamental computational block of multilayer, fully-connected, feedforward artificial neural networks, which are also commonly, but misleadingly, called multilayer perceptrons (MLPs) [28]. MLPs constitute essential submodules within many contemporary architectures. These networks are trained end-to-end for classification, regression, generation or representation learning via backpropagation [29, 30]. Standard ANNs process static input vectors and compute continuous activations layer by layer, overlooking the asynchronous spike-timing code used by biological neurons.

3.2 | SPIKING NEURONS AND THE SPIKE-RESPONSE MODEL

Spiking neural networks (SNNs) exploit discrete, all-or-none spikes as the primary events driving sparse, event-driven computation [31, 32]. Each neuron integrates incoming synaptic inputs into its membrane potential, emits an instantaneous spike when that potential crosses a threshold, and then resets.

More generally, the membrane potential dynamics of a spiking neuron may be represented by a sum of causal kernels in the framework of the Spike-Response Model (SRM) [18, 33]:

$$V(t) = V_{\text{rest}} + \sum_j \sum_{t_j^{(f)} < t} w_j \varepsilon(t - t_j^{(f)}) + \sum_{t^{(f)} < t} \eta(t - t^{(f)}), \quad (3.2)$$

where V_{rest} is the resting membrane potential, $\varepsilon(s)$ is the postsynaptic kernel, $\eta(s)$ is the refractory kernel (with $\eta(0) = V_{\text{reset}} - V_{\text{th}}$), and $t^{(f)}$ are the neuron's own spike times, by construction $\varepsilon(s) = \eta(s) = 0$ for $s < 0$, so that each convolution is causal. During the subthreshold regime (i.e., between its own output spikes), $V(t)$ evolves linearly via these convolutions, and whenever $V(t^{(f)-}) \geq V_{\text{th}}$ at some $t = t^{(f)}$ the neuron fires and $V(t^{(f)+}) = V(t^{(f)-}) + \eta(0) = V_{\text{reset}}$.

The neuron's output spike train is

$$y(t) = \sum_f \delta(t - t^{(f)}),$$

where each $t^{(f)}$ are the neuron's output spike times. To focus on computation, define the input and output spaces

$$\mathcal{X} = \left\{ \mathbf{x}(t) = (x_1(t), \dots, x_d(t)) \mid x_j(t) = \sum_f \delta(t - t_j^{(f)}) \right\}, \quad \mathcal{Y} = \left\{ y(t) = \sum_f \delta(t - t^{(f)}) \right\}.$$

Since each spike train d is a sum of Dirac deltas, convolving a kernel k with that train reproduces exactly the corresponding sums:

$$(k * d)(t) = \int k(t - s) d(s) ds = \int k(t - s) \sum_f \delta(s - t^{(f)}) ds = \sum_{t^{(f)} < t} k(t - t^{(f)}).$$

Hence we can write

$$(\varepsilon * x_j)(t) = \sum_{t_j^{(f)} < t} \varepsilon(t - t_j^{(f)}), \quad (\eta * y)(t) = \sum_{t^{(f)} < t} \eta(t - t^{(f)}).$$

The membrane-potential $V(t)$ then evolves as

$$V(t) = V_{\text{rest}} + \sum_{j=1}^d w_j (\varepsilon * x_j)(t) + (\eta * y)(t). \quad (3.3)$$

Each neuron thus implements

$$s_{\mathbf{w}, V_{\text{rest}}, V_{\text{th}}} : \mathcal{X} \rightarrow \mathcal{Y},$$

where, given $\mathbf{x}(t) \in \mathcal{X}$, the output spike train is generated according to the thresholding and reset rules. Collecting all parameters defines the class

$$\mathcal{S}_{\varepsilon, \eta} = \left\{ s_{\mathbf{w}, V_{\text{rest}}, V_{\text{th}}} \mid \mathbf{w} \in \mathbb{R}^d, V_{\text{rest}}, V_{\text{th}} \in \mathbb{R} \right\}.$$

In the Spike-Response Model (SRM), the postsynaptic and refractory kernels $\varepsilon(s)$ and $\eta(s)$ encode how input and output spikes shape the membrane potential [32, 34]. Building on this foundation, we observe that the analytic properties of the kernels directly shape the temporal structure of the induced function class $\mathcal{S}_{\varepsilon, \eta}$, and thereby govern the expressivity, trainability, and inductive biases of spike-based models [35, 36, 37, 38, 39].

The tuple $(V_{\text{rest}}, \{w_j\}, \varepsilon, \eta, V_{\text{th}})$ specifies one spiking neuron. Such neurons are assembled into layers or recurrent networks, and trained via surrogate-gradient, event-driven, or local plasticity rules for tasks including temporal pattern recognition, classification, and control [40, 41, 42, 43, 44, 45].

A widely used neuronal model in machine learning is the Leaky Integrate-and-Fire (LIF). By choosing different postsynaptic and refractory kernels, the SRM can emulate a variety of spiking-neuron dynamics including the LIF neuron (where one chooses exponential kernels $\varepsilon(s) = e^{-s/\tau_m}$, $\eta(s) = (V_{\text{reset}} - V_{\text{th}}) e^{-s/\tau_{\text{ref}}}$).

A LIF neuron's membrane potential $V(t)$ evolves according to the RC circuit model of a passive membrane [18, 32],

$$\begin{aligned}\tau_m \frac{dV}{dt} &= -(V(t) - V_{\text{rest}}) + R_m I_{\text{syn}}(t), \\ I_{\text{syn}}(t) &= \sum_j w_j \sum_{t_j^{(f)} < t} \delta(t - t_j^{(f)}),\end{aligned}\tag{3.4}$$

where $I_{\text{syn}}(t)$ is the synaptic input current, $\tau_m = R_m C_m$ is the membrane time constant, with R_m is the membrane resistance and C_m is the membrane capacitance, V_{rest} is the resting potential, w_j is the synaptic weight on afferent j , $t_j^{(f)}$ are presynaptic spike times, and δ is the Dirac delta.

In SNNs, real-valued inputs are typically encoded as either rate codes or temporal codes. In a rate code, the mean firing rate over a fixed window is proportional to the input; common implementations use Poisson or periodic spike trains. In a temporal code, information is carried by precise spike timing or order (for example time-to-first-spike, phase-within-oscillation, or inter-spike-interval schemes).

3.3 | ANN-SNN CORRESPONDENCE

Assuming that both inputs and outputs are represented only by firing rates (with no timing information), the SNN's average input-output mapping reduces to a static weighted sum followed by a nonlinearity, which is essentially the same as a standard perceptron activation. Under standard rate-coding and steady-state assumptions, a single SRM neuron likewise reduces to a perceptron acting on input firing rates. This forms the basis for the ANN-SNN correspondence used for weight conversion and layer approximation, which we detail in the following paragraphs.

Assume each input channel's Poisson rate is held constant over a window long enough for the output to reach a steady firing rate. Let each input channel j emit an independent, homogeneous Poisson spike train,

$$x_j(t) = \sum_f \delta(t - t_j^{(f)}), \quad \mathbb{E}[x_j(t)] = r_j.$$

Define the kernel integrals

$$I_\varepsilon \equiv \int_0^\infty \varepsilon(s) ds, \quad I_\eta \equiv \int_0^\infty \eta(s) ds.$$

Assume ε and η are absolutely integrable on $[0, \infty)$, i.e. $\int_0^\infty |\varepsilon(s)| ds < \infty$ and $\int_0^\infty |\eta(s)| ds < \infty$, which ensures these integrals converge. Then by linearity of convolution and using the Poisson rates r_j and output rate F_y

$$\mathbb{E}[(\varepsilon * x_j)(t)] = r_j \int_0^\infty \varepsilon(s) ds = r_j I_\varepsilon, \quad \mathbb{E}[(\eta * y)(t)] = F_y \int_0^\infty \eta(s) ds = F_y I_\eta.$$

Hence the mean membrane potential is

$$\bar{V} = V_{\text{rest}} + \sum_{j=1}^d w_j r_j I_\varepsilon + F_y I_\eta.\tag{3.5}$$

A SRM neuron fires each time $V(t)$ crosses threshold V_{th} , then resets. Under stationarity and ergodicity, so that $\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T u(t) dt = \mathbb{E}[u(t)]$, one can approximate the input-output transfer by a smooth function g , so that

$$F_y \approx g(\bar{V} - V_{\text{th}}).\tag{3.6}$$

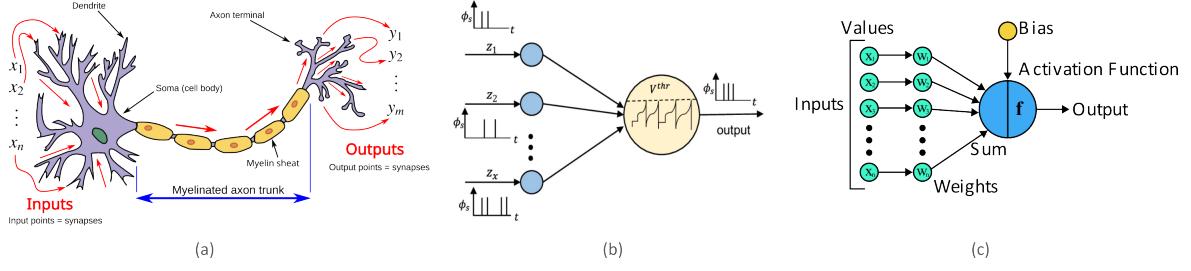


Figure 3.1: From realistic neuronal models to abstractions: (a) biological neuron [52]; (b) spiking neuron [53]; (c) perceptron.

In many cases (e.g. low-noise regime, exponential kernels, moderate thresholds), g is a rectified-linear or sigmoid-like nonlinearity. Substituting $\bar{V}$ gives

$$F_y \approx g \left(V_{\text{rest}} - V_{\text{th}} + \sum_j w_j r_j I_\varepsilon + F_y I_\eta \right).$$

If the refractory integral I_η is negligible (or folded into g), this simplifies to

$$F_y \approx g \left(\sum_j w_j I_\varepsilon r_j + (V_{\text{rest}} - V_{\text{th}}) \right). \quad (3.7)$$

Defining

$$w_{\text{eff},j} = w_j \int_0^\infty \varepsilon(s) ds, \quad b_{\text{eff}} = V_{\text{rest}} - V_{\text{th}}, \quad \phi(u) = g(u).$$

For each channel j , collect the input rates and effective weights into vectors

$$\mathbf{x} = (r_1, \dots, r_d)^\top, \quad \mathbf{w}_{\text{eff}} = (w_{\text{eff},1}, \dots, w_{\text{eff},d})^\top.$$

Then the SRM's steady-state output firing rate satisfies

$$F_y \approx \phi \left(\mathbf{w}_{\text{eff}}^\top \mathbf{x} + b_{\text{eff}} \right). \quad (3.8)$$

which is algebraically identical to the perceptron mapping with weights $\mathbf{w}_{\text{eff}}$, bias b_{eff} , and activation ϕ (but only under strong simplifying assumptions). In other words, the typical artificial perceptron can be thought of as a rate-based abstraction of a spiking neuron (Figure 3.1), capturing its essential “sum-and-threshold” behaviour in a static, continuous form [19, 46]. This correspondence supports the intuition that trained ANN weights can be converted into SNN synaptic weights by matching firing-rate curves to activation functions, and that SNNs can approximate ANN layer computations under suitable encoding and decoding schemes [47, 48, 49, 50, 51].

* * *

In chapter 6, we will develop a clearer understanding of the motivation for establishing a *soft equivalence* between perceptrons and rate-coded spiking neurons, because our photonic quantum neuron framework renders their distinction ambiguous.

FROM PERCEPTRONS TO NEURAL NETWORKS

This chapter formalizes the transition from a single perceptron to MLPs. We describe feedforward composition and summarize essential results on expressivity and training; broader architectural variants and generalization theory.

4.1 | FEEDFORWARD NEURAL COMPOSITION

A depth- L MLP (Figure 4.1) with widths $(n_0 = d, n_1, \dots, n_L)$ repeats two steps: an *affine mixing* of the current features, followed by an *elementwise nonlinearity*. For input $\mathbf{x} \in \mathbb{R}^{n_0}$ and layers $\ell = 1, \dots, L$,

$$\mathbf{a}^{(\ell)} = \phi_\ell(W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \mathbf{a}^{(0)} = \mathbf{x}.$$

The intermediate vector $W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ is the *preactivation*; $\mathbf{a}^{(\ell)}$ is the activation passed forward. Collectively, the network implements a composition of simple transformations: affine maps $\mathcal{A}_{W^{(\ell)}, \mathbf{b}^{(\ell)}}(\cdot) = W^{(\ell)}(\cdot) + \mathbf{b}^{(\ell)}$ interleaved with elementwise nonlinearities ϕ_ℓ .

Elementwise nonlinearities are crucial: without them, stacked affine maps collapse to a single affine map and cannot model nonlinear relations. Different activations shape smoothness, sparsity, and saturation, affecting both optimization and inductive bias.

This composition view explains why stacking matters. A single perceptron is a detector that scores input features and then “squashes” the score. A *layer* is many such detectors in parallel, each attending to a different aspect of the input. Width provides *coverage* (more parallel channels at a given level of abstraction). *Depth* then builds *features of features*: early layers respond to simple, local regularities; later layers combine these responses into progressively more abstract concepts (e.g., edges $\rightarrow$ corners $\rightarrow$ shapes $\rightarrow$ objects; characters $\rightarrow$ subwords $\rightarrow$ words $\rightarrow$ topics/sentiment). Bias terms shift the detectors’ thresholds, making them sensitive to patterns regardless of absolute scale and easing downstream recombination. The hierarchy—built entirely from repeatable mix–squash steps—is the source of deep networks’ expressive power.

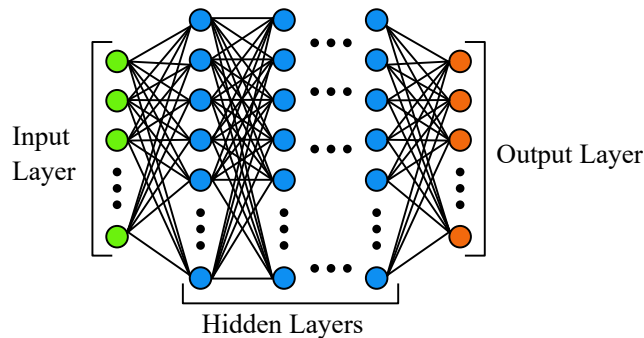


Figure 4.1: Multilayer perceptron; a fully connected, feedforward network. Edges denote perceptrons and information flows left→right without recurrence.

A single perceptron draws only linear decision boundaries; classic counterexamples such as XOR require constructing a useful nonlinear feature map before a final linear readout. Even a shallow MLP with one hidden layer achieves this by learning intermediate detectors whose combinations become linearly separable at the output.

4.2 | EXPRESSIVITY AND GENERALIZATION

A single hidden layer MLP with a nonpolynomial activation can approximate any continuous function on a compact set given sufficient width. Width alone thus buys *universality* [54, 55]. However, many target functions admit far more *compact* (parameter-efficient) representations when depth mirrors their inherent compositional structure. Depth-separation results formalize this advantage: certain functions require exponentially many units in shallow architectures but only polynomially many when depth is available [56]. Stacking layers therefore provides expressive structure beyond merely adding parameters.

Depth, width, and activation collectively define the architecture and its inductive biases, i.e., which structures are represented naturally (with fewer parameters and less data). Residual (skip) connections let layers learn perturbations of the identity, aiding gradient flow and enabling very deep stacks [57]. Normalization layers (batch or layer norm) stabilize intermediate statistics and smooth the loss landscape, accelerating training, while proper initialization preserves signal scale across layers and prevents exploding or vanishing gradients [58].

4.3 | LEARNING OBJECTIVES AND OPTIMIZATION

Given data $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, we choose parameters $\theta = \{W^{(\ell)}, \mathbf{b}^{(\ell)}\}$ by minimizing an objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i) + \lambda \mathcal{R}(\theta). \quad (4.1)$$

Here ℓ measures per-example discrepancy between prediction and target, and $\mathcal{R}$ is a preference for simpler or structured parameters (e.g., ℓ_2 “weight decay”). The scalar $\lambda \geq 0$ trades off fit and simplicity; $\lambda = 0$ recovers unregularized empirical risk minimization. Typical choices are mean-squared error or cross-entropy for ℓ and $\|\ell\|_2$ or $\|\ell\|_1$ penalties for $\mathcal{R}$.

Task-specific output layers and losses are standard: identity + mean-squared error for regression; sigmoid + logistic loss for binary classification; softmax + cross-entropy for multi-class classification.

Reverse-mode automatic differentiation (*backpropagation*) computes gradients efficiently by applying the chain rule from the output layer back to the input. Stochastic gradient methods (SPSA, SGD with momentum, Adam, RMSprop) update parameters using minibatches [28].

The parameter count scales as $\sum_{\ell=1}^L (n_{\ell} n_{\ell-1} + n_{\ell})$, and the dominant per-pass compute is $O(\sum_{\ell} n_{\ell} n_{\ell-1})$. Moving from a perceptron to an L -layer MLP increases capacity and compute roughly linearly in depth and quadratically in widths.

4.4 | QUANTUM NEURAL NETWORKS

In quantum computing, models commonly called *quantum neural networks* (QNNs) are most typically understood as *parameterized quantum circuits* (PQCs) trained in a hybrid quantum–classical loop (Figure 4.2), rather than as stacks of quantum “neurons.” A typical supervised model encodes an input x into a quantum state, applies a trainable circuit, and measures an observable to produce a scalar or vector output:

$$\rho_{\theta}(x) = U(\theta) U_{\text{enc}}(x) \rho_0 U_{\text{enc}}(x)^{\dagger} U(\theta)^{\dagger}, \quad f_{\theta}(x) = \text{Tr}[M \rho_{\theta}(x)].$$

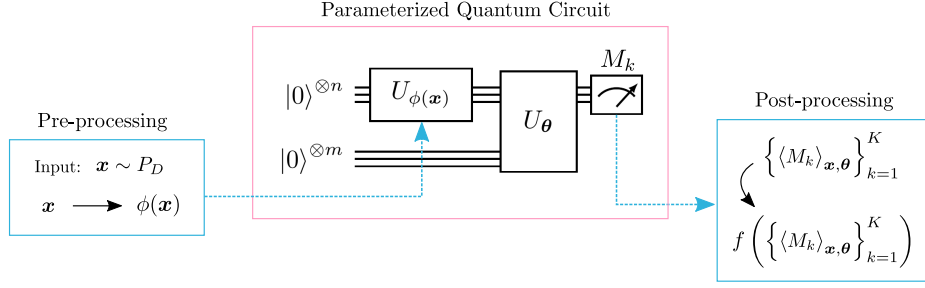


Figure 4.2: A parameterized quantum circuit embedded in a classical pre- and postprocessing workflow [64].

Here $U_{\text{enc}}(x)$ is the data-encoding unitary, $U(\theta)$ the trainable ansatz, $\rho_{\theta}(x)$ a density operator (with ρ_0 the fixed reference state), and M the measurement operator [59, 60]. Learning proceeds by estimating $f_{\theta}(x)$ (via shots on hardware or simulation), evaluating a loss, and updating θ using classical optimization.

Aspects of the model specification that largely determine its behaviour include the encoder (which fixes the induced feature map and its locality), the ansatz (which sets the entanglement structure and symmetries), and the measurement strategy (which governs signal-to-noise and evaluation cost). Although the circuit’s internal evolution is linear and unitary, effective nonlinearity enters through measurement and, where available, mid-circuit measurement with feed-forward. Consequently, “layers” in QNNs are circuit blocks of native gates rather than elementwise activations acting independently on wires.

When the circuit is fixed, the model reduces to a (quantum) kernel method: encoding $x \mapsto \rho(x)$ induces a kernel $k(x, x') = \text{tr}[\rho(x)\rho(x')]$, and training amounts to a linear model in that feature space evaluated with a quantum device [59]. Allowing θ to vary co-learns the feature map and the decision rule. In wide or small-update regimes the training dynamics can approach kernel regression governed by a quantum neural tangent kernel, clarifying when a PQC behaves as a fixed feature projector rather than a feature learner [61].

Practical performance on near-term hardware is shaped by two constraints. First, *barren plateaus*: for broad families of random deep circuits and global observables, gradient variance can decay exponentially in qubit number, impeding training [62]. Empirically supported mitigations include locality and structure in the ansatz, symmetry preservation, layerwise growth, and metric-aware updates (e.g., quantum natural gradient) [60, 63]. Second, noise and finite sampling: limited fidelity and shot budgets penalize depth and large measurement sets, motivating shallow layouts and measurement-efficient readouts [60].

It follows that QNNs are not straightforward quantum analogues of multilayer perceptrons. Quantum evolution prohibits the classical fan-out and elementwise nonlinear activations that underwrite perceptron stacks; the useful inductive biases in QNNs are imposed by data encoding, entanglement pattern, symmetries, and readout. There is no universally accepted “quantum perceptron” playing the role of a classical neuron. The next chapter examines proposed *quantum perceptron* models and their abstractions, focusing on what they preserve from the classical picture and what must change to respect quantum mechanics.

QUANTUM PERCEPTRONS

The quantum perceptron abstraction developed in this section draws on the classic references of quantum information theory [65, 66, 67, 68].

The earliest ideas in quantum neural networks emerged in the mid-1990s, with independent contributions such as the concept of a self-reconfiguring quantum neural computer [69], quantum-mechanical implementations of adaptive learning machines [70], and, most relevant to our discussion, a statistical theory of quantum perceptron units [71]. Subsequent efforts have refined and extended these models, proposing a variety of theoretical frameworks for quantum neurons [72, 73].

A quantum perceptron implements a quantum analog of the perceptron mapping, defined through several mathematical structures [74, 75]. We distill the shared logical structure of existing quantum-neuron models using the tools of quantum information theory.

5.1 | QUANTUM PERCEPTRON MODEL

A quantum perceptron acts on logical subsystems for input, weights, ancilla, and output:

$$\begin{aligned}
 \mathcal{H}_x &= \bigotimes_{j=1}^{N_x} \mathcal{H}_{d_j}, & \dim \mathcal{H}_x &= \prod_{j=1}^{N_x} d_j = D_x, \\
 \mathcal{H}_w &= \bigotimes_{k=1}^{N_w} \mathcal{H}_{d_k}, & \dim \mathcal{H}_w &= \prod_{k=1}^{N_w} d_k = D_w, \\
 \mathcal{H}_a, & & \dim \mathcal{H}_a &= D_a, \\
 \mathcal{H}_{\text{out}} &= \mathcal{H}_y \otimes \underbrace{\text{span}\{|y\rangle\}_{y \in \mathcal{Y}}}_{\mathcal{H}_{\text{cl}}}, & \dim \mathcal{H}_{\text{out}} &= \underbrace{D_y}_{\mathcal{H}_y} \underbrace{|\mathcal{Y}|}_{\mathcal{H}_{\text{cl}}}
 \end{aligned}$$

Here each d_j is either a finite integer ≥ 2 (e.g., a qudit of dimension d_j) or $= \infty$ for continuous-variable modes.

A classical label register with alphabet $\mathcal{Y}$ is

$$\mathcal{H}_{\text{cl}} = \text{span}\{|y\rangle : y \in \mathcal{Y}\},$$

with $\{|y\rangle\}_{y \in \mathcal{Y}}$ an orthonormal basis; classical states are precisely those density operators that are diagonal in this basis.

The logical workspace and total circuit spaces at the register level are

$$\underbrace{\mathcal{H}_x \otimes \mathcal{H}_w}_{\text{logical QP space } \mathcal{H}} \xleftarrow{\text{Tr}_a} \underbrace{\mathcal{H}_x \otimes \mathcal{H}_w \otimes \mathcal{H}_a}_{\text{processing workspace } \mathcal{H}_{\text{proc}}} \xleftarrow{\text{Tr}_{\text{out}}} \underbrace{\mathcal{H}_x \otimes \mathcal{H}_w \otimes \mathcal{H}_a \otimes \mathcal{H}_{\text{out}}}_{\text{full circuit space } \mathcal{H}_{\text{tot}}}.$$

States on a quantum register R with Hilbert space $\mathcal{H}_R$ form the state space $\mathcal{S}(\mathcal{H}_R)$ of density operators

$$\mathcal{S}(\mathcal{H}_R) = \{\rho \in \mathcal{T}(\mathcal{H}_R) : \rho \geq 0, \text{Tr } \rho = 1\}, \quad (5.1)$$

where $\mathcal{T}(\mathcal{H}_R)$ denotes trace-class operators; in finite dimension this coincides with $\mathcal{L}(\mathcal{H}_R)$, the bounded linear operators on $\mathcal{H}_R$. In what follows we work in finite dimension and write everything on $\mathcal{L}(\cdot)$; for infinite-dimensional/continuous-variable settings one may replace $\mathcal{L}(\cdot)$ by the trace-class $\mathcal{T}(\cdot)$ with some adjustments.

A rich variety of encoding strategies have been developed. Binary inputs and weights can be embedded in multi-qubit superpositions [75], while continuous-valued qubit amplitudes enable smooth, differentiable maps [76]. Input-conditioned ancilla operations combined with phase estimation could provide an encoding route for multidimensional, potentially non-linearly separable inputs [77]. For weight encoding, constraining parameters to real values removes phase degrees of freedom and stabilises hybrid training loops [78]. Synaptic weights can also reside in quantum degrees of freedom that co-evolve with neuron states, realising Hebbian plasticity [79].

Classical data $x \in \mathcal{X} \subset \mathbb{R}^d$ and weights $w \in \mathcal{W} \subset \mathbb{R}^s$ are loaded into quantum states by two completely positive, trace-preserving (CPTP) maps

$$\mathcal{E}_x : \mathcal{L}(\mathcal{H}_x) \rightarrow \mathcal{L}(\mathcal{H}_x), \quad \mathcal{E}_w : \mathcal{L}(\mathcal{H}_w) \rightarrow \mathcal{L}(\mathcal{H}_w), \quad (5.2)$$

so that for fixed references $\rho_0 = |0\rangle\langle 0|^{\otimes n}$, $\sigma_0 = |0\rangle\langle 0|^{\otimes m}$,

$$\rho_x = \mathcal{E}_x(\rho_0) = \sum_{\alpha} K_{\alpha}(x) \rho_0 K_{\alpha}(x)^{\dagger}, \quad \sigma_w = \mathcal{E}_w(\sigma_0) = \sum_{\beta} K_{\beta}(w) \sigma_0 K_{\beta}(w)^{\dagger}, \quad (5.3)$$

where $\{K_{\alpha}, K_{\beta}\}$ are Kraus operators satisfying $\sum_{\alpha} K_{\alpha}^{\dagger} K_{\alpha} = 1$, $\sum_{\beta} K_{\beta}^{\dagger} K_{\beta} = 1$. In the unitary, noise-free limit these reduce to familiar feature encodings: $\rho_x = U_x \rho_0 U_x^{\dagger}$, $\sigma_w = U_w \sigma_0 U_w^{\dagger}$. Familiar choices include amplitude, phase, angle and basis encodings or, in continuous-variable (CV) settings, optical displacements.

The quantum perceptron's forward pass is the composition

$$\mathcal{G} = \mathcal{R} \circ \mathcal{F}, \quad (5.4)$$

where the inference channel

$$\mathcal{F} : \mathcal{L}(\mathcal{H}_x \otimes \mathcal{H}_w) \longrightarrow \mathcal{L}(\mathcal{H}_x \otimes \mathcal{H}_w) \quad (5.5)$$

acts on $\rho_x \otimes \sigma_w$, and the read-out instrument

$$\mathcal{R} : \mathcal{L}(\mathcal{H}_x \otimes \mathcal{H}_w) \rightarrow \mathcal{L}(\mathcal{H}_{\text{out}}) \quad (5.6)$$

produces a quantum/classical result.

At the inference stage, implementations range from reversible, unitary evolution to dissipative/open-system dynamics. Dissipative models can reach useful chaotic regimes [80]. In the unitary case, the inference channel is a purely reversible unitary on inputs and ancilla, and arbitrary multi-qubit unitaries provide universal expressivity [74, 81]. Augmenting the workspace with internal-memory qubits yield recurrent behaviour reminiscent of classical gated units [82]. Other mechanisms exploit familiar quantum-subroutine techniques such as Grover-based amplitude amplification for perceptron updates [83], and the superposition-architecture search operator [84]. Beyond circuit models, wave-based proposals perform the weighted sum via linear interference, while local intensity-dependent index changes implement Hebbian weight updates [85]. A many-body view treats the perceptron as a quantum particle in a random potential; learning then means finding low-energy states [86].

By Stinespring's theorem, the inference channel can always be implemented by a unitary U_{xwa} acting on the enlarged space $\mathcal{H}_x \otimes \mathcal{H}_w \otimes \mathcal{H}_a$ followed by a partial trace over $\mathcal{H}_a$,

$$\mathcal{F}[\rho] = \text{Tr}_a [U_{xwa} (\rho_{xw} \otimes \rho_a) U_{xwa}^\dagger] = \sum_Y K_Y \rho K_Y^\dagger,$$

with unitaries U_{xwa} on the extended space and Kraus operators K_Y . Let $J_{\mathcal{F}}$ be the Choi matrix of $\mathcal{F}$; one may always choose

$$D_a = \text{rank}\{J_{\mathcal{F}}\} \leq (\dim \mathcal{H})^2 = (D_x D_w)^2,$$

and in practice an r -Kraus implementation needs at least $D_a \geq r$ distinct ancilla levels.

The read-out instrument is realized by

$$\mathcal{R}[\rho] = \sum_{y,\mu} (K_{y,\mu} \rho K_{y,\mu}^\dagger) \otimes |y\rangle\langle y| \quad (5.7)$$

so that tracing out $\mathcal{H}_{\text{cl}}$ yields a post-measurement quantum state in $\mathcal{H}_y$, while tracing out $\mathcal{H}_y$ yields classical label probabilities $\Pr(y)$, and computing $\sum_y y \Pr(y)$ gives expectation values.

Activation and readout mechanisms instantiate non-affinity in several ways. On NISQ hardware, constant-depth variational circuits and small, polynomial-depth subcircuits approximate complex nonlinear maps and classical activations such as sigmoid or ReLU [87, 88]. Measurement-based realizations include projective thresholding via quantum phase estimation [89], probabilistic heralding through Repeat-Until-Success primitives [90], and swap tests with phase estimation for state-overlap thresholding [91]. Similar effects arise by tracing out subsystems [92] or by coupling to engineered reservoirs [93, 94]. Algebraic deformations offer another route to non-classical statistics [95]. In the continuous-variable regime, Gaussian modes with Kerr nonlinearities realize ReLU-style activations [96]. Adiabatic Ising ramps yield smooth sigmoid like responses [97], with shortcuts to adiabaticity accelerating these responses [98].

5.2 | NONLINEARITY AND ACTIVATIONS

A quantum perceptron's "activation" is any operation that renders the end-to-end input-output map non-affine. A map $f : X \rightarrow Y$ is non-affine if it fails to preserve some affine combination, i.e.,

$$\exists x_1, x_2 \in X, k \in \mathbb{R} \quad \text{such that} \quad f((1-k)x_1 + kx_2) \neq (1-k)f(x_1) + kf(x_2). \quad (5.8)$$

Non-affinity can be introduced with the following mechanisms:

State-space non-affinity. In particular, let $X = \mathcal{S}(\mathcal{H}_x)$ and $Y = \mathcal{S}(\mathcal{H}_{\text{out}})$, and let $\mathcal{M}$ be a completely positive trace non-increasing (CPTNI) map and not CPTP, and define

$$p_i = \Pr_{\mathcal{M}}(\text{accept} \mid \rho_i) = \text{Tr}[\mathcal{M}(\rho_i)], \quad C_{\mathcal{M}}(\rho) = \frac{\mathcal{M}(\rho)}{\Pr_{\mathcal{M}}(\text{accept} \mid \rho)}. \quad (5.9)$$

Since $\mathcal{M}$ is not trace-preserving, there exist ρ_1, ρ_2 and $k \in (0, 1)$ such that

$$C_{\mathcal{M}}((1-k)\rho_1 + k\rho_2) = \frac{\mathcal{M}((1-k)\rho_1 + k\rho_2)}{(1-k)p_1 + kp_2} \quad (5.10)$$

$$\neq (1-k) \frac{\mathcal{M}(\rho_1)}{p_1} + k \frac{\mathcal{M}(\rho_2)}{p_2} \quad (5.11)$$

$$= (1-k)C_{\mathcal{M}}(\rho_1) + kC_{\mathcal{M}}(\rho_2). \quad (5.12)$$

Hence $f = C_{\mathcal{M}}$ fails the affine condition and succeeds only probabilistically.

These activations include:

- **Measurement-based activations:**

Projective/POVM post-selection:

$$\rho \mapsto \frac{M_y \rho M_y^\dagger}{\text{Tr}[M_y \rho]}.$$

Repeat-Until-Success (RUS): given success operator M_s , failure operator M_f , and recovery unitary U , the conditional RUS channel is

$$C_{\text{RUS}}(\rho) = \frac{\sum_i (M_f U)^i M_s \rho M_s^\dagger (U^\dagger M_f^\dagger)^i}{\text{Tr} \left[\sum_i (M_f U)^i M_s \rho M_s^\dagger (U^\dagger M_f^\dagger)^i \right]},$$

which iterate unitaries and measurements until a designated outcome heralds the desired CPTNI map (threshold, sigmoid, ReLU, polynomials).

- **Dissipative/trace-out activations:**

Heralded partial trace: where one projects an ancilla onto a “success” subspace Π and discards the rest,

$$\rho_S \mapsto \frac{\text{Tr}_A[(1 \otimes \Pi) U (\rho_S \otimes |0\rangle\langle 0|) U^\dagger]}{\text{Tr}[(1 \otimes \Pi) U (\rho_S \otimes |0\rangle\langle 0|) U^\dagger]}.$$

Engineered, heralded dissipation: given Lindblad jump operators $\{L_j\}$ and a subset S of “heralded” jumps,

$$\mathcal{E}_S(\rho) = \sum_{j \in S} L_j \rho L_j^\dagger, \quad C_S(\rho) = \frac{\mathcal{E}_S(\rho)}{\text{Tr}[\mathcal{E}_S(\rho)]}.$$

Parameter-space non-affinity. In particular, let $X \subseteq \mathbb{R}^n$, $Y \subseteq \mathbb{R}^m$, $\rho_{\text{in}} \in \mathcal{S}(\mathcal{H})$, fix a POVM $\{M_y\}_{y=1}^m$, a projector (or POVM element) P , and a CPTP family $\{\Lambda_x\}_{x \in X}$. Write

$$\mathcal{P}_m = \left\{ p \in \mathbb{R}^m : p_y \geq 0, \sum_{y=1}^m p_y = 1 \right\}, \quad p(x)_y = \text{Tr}[M_y \Lambda_x(\rho_{\text{in}})], \quad \langle O \rangle_x = \text{Tr}[O \Lambda_x(\rho_{\text{in}})],$$

$$c(x) = p(x) \text{ or } \langle O \rangle_x, \quad \mathcal{C} = \mathcal{P}_m \cup \left\{ \langle O \rangle_x : x \in X \right\} \subset \mathbb{R}^m \cup \mathbb{R}.$$

Let d be the number of real control parameters in a data-controlled unitary. Then any map $f : X \rightarrow Y$ of the form

$$f(x) = \begin{cases} \text{Tr}[P U(\phi(x)) \rho_{\text{in}} U(\phi(x))^\dagger], & \phi : X \rightarrow \mathbb{R}^d \text{ nonlinear (feature encoding),} \\ \varphi(c(x)), & \varphi : \mathcal{C} \rightarrow Y \text{ nonlinear (classical post-processing),} \end{cases} \quad (5.13)$$

fails the affine-combination test because in each branch the underlying classical map (ϕ or φ) does not preserve some convex combination.

These include instances:

- **Nonlinear feature encodings:**

Phase-kickback circuits:

$$U(\phi(x)) = \exp[i H \phi(x)], \quad f(x) = \text{Tr}[P e^{iH\phi(x)} \rho_{\text{in}} e^{-iH\phi(x)}].$$

Amplitude-based encodings: Prepare $|\psi(x)\rangle = V(\phi(x))|0\rangle$, then

$$f(x) = \langle \psi(x) | O | \psi(x) \rangle.$$

General parametrized rotations:

$$U(\phi(x)) = \prod_{j=1}^d e^{-i \phi_j(x) H_j}, \quad f(x) = \text{Tr}[P U(\phi(x)) \rho_{\text{in}} U(\phi(x))^\dagger].$$

- **Classical post-processing:**

Nonlinear functions on outcome probabilities: $f(x) = \varphi(p(x))$.

Expectation-value post-processing: $f(x) = \varphi(\langle O \rangle_x)$.

5.3 | FUNCTION AND PROPERTIES

We summarize the action of a quantum perceptron as the tuple of maps

$$(\mathcal{E}_x, \mathcal{E}_w, \mathcal{G}), \quad \text{with function class } \mathcal{Q} = \{f_w : x \mapsto \mathcal{G}[\mathcal{E}_x \otimes \mathcal{E}_w]\}. \quad (5.14)$$

The analytic properties of $(\mathcal{E}_x, \mathcal{E}_w, \mathcal{G})$ determine the behaviour of quantum models:

- *Expressivity.* The input encoder $\mathcal{E}_x$ and weight encoder $\mathcal{E}_w$ lift classical data and parameters into a high-dimensional quantum feature space, while the channel $\mathcal{G}$ introduces the necessary “quantum nonlinearity.” Together they determine how rich a family of decision boundaries (or functions) the perceptron can carve out [59, 99].
- *Trainability.* The same freedom that gives rich representations can also flatten the optimization landscape (“barren plateaus”), making gradients tiny [62]. Shallow or structured encodings, local measurements, engineered dissipation and smart initialization help maintain non-vanishing gradients and keep learning feasible [63, 100, 101].
- *Inductive bias.* The data encoding $\mathcal{E}_x$ implicitly defines which patterns are easiest to distinguish, much like a kernel in classical learning [61]. Choices in $\mathcal{E}_w$ and $\mathcal{G}$ then further favour or penalize particular solution classes, guiding the model toward plausible hypotheses before seeing any data [60, 102, 103].

* * *

Practical realizations of quantum perceptrons emphasise simplicity and demonstrations on current hardware. Even single-qubit rotations with probabilistic read-out now serve as practical perceptrons, enabling lightweight models suitable for near-term hardware [104]. Recent gate-model constructions extend this idea, showing that the truncated Taylor series of any analytic activation can be encoded coherently, without intermediate measurements, so that a single layer already satisfies universal-approximation conditions [105]. Experimental demonstrations include trapped-ion schemes using adiabatically ramped dressing fields to realise smooth sigmoid gates [106], and Rydberg-atom arrays exploiting blockade interactions for single-shot, multiclass, error-tolerant perceptrons [107].

Neuromorphic approaches aim to reproduce spiking behaviour within quantum dynamics. Hamiltonian evolution followed by projective read-out produces threshold firing with measurement back-action [108]. Leaky integrate-and-fire behaviour is obtained by adding controlled dissipation [109]. A quantised Hodgkin–Huxley model predicts superposed and entangled action potentials [110]. Quantum-inspired classical models reproduce superposition-like parallel spike accumulation prior to thresholding [111].

PHOTON-NATIVE QUANTUM PERCEPTRON

We work in the photon-native algorithmic regime. The essential primitives were reviewed in [chapter 2](#) and are not repeated here.

Throughout this chapter and the remainder of the report, we adopt the working intuition that photon occupation is analogous to a spike. As discussed in [chapter 3](#), spiking information may be rate coded. Because we do not employ time bins, occupation across spatial modes serves as the effective rate code.

6.1 | DATA ENCODING AS UNITARY FEATURE MAPS

Let us begin with the basic problem: embedding classical data into a quantum system. Standard schemes include amplitude encoding, phase encoding, and hybrids of the two.

We formalize encoding as a unitary feature map $U(x)$ acting on a reference state. A convenient special case arises when the mode associated with an input x_i carry amplitudes proportional to its rate, so that $|a_i|^2$ reflects the corresponding event rate. This aligns with our definition of a spike for the photon-native quantum perceptron (PnQP). Phase information can also be used, of course.

From this generalized viewpoint the encoding should be task dependent. Nevertheless, a practical choice is a Householder-based scheme. Let $\tilde{x} = x/\|x\|$ and e_1 be the first basis vector. Define

$$v = e_1 - \tilde{x}, \quad H(x) = I - 2 \frac{vv^\dagger}{v^\dagger v}. \quad (6.1)$$

Then $H(x) e_1 = \tilde{x}$, so a single unitary prepares amplitudes proportional to the normalized data.

Another common option is diagonal phase encoding implemented by modewise phase shifters,

$$U_\phi(x) = \exp(i \text{diag}(\phi(x))), \quad (6.2)$$

which imprints input-dependent phases on the designated modes.

6.2 | UNITARY-PARAMETRIZED AFFINE MAP

We now address how to implement a weighted sum, or more generally an affine map, of the inputs.

A minimal analogue of a classical perceptron uses two unitary blocks: an encoding $U_{\text{enc}}(x)$ and a weight unitary U_w . Prepare an input Fock state with photons in designated data modes, apply $U = U_w U_{\text{enc}}(x)$, and read out the expected photon number on a chosen output mode m . With mode creation operators transforming as $a_j^\dagger \mapsto \sum_k U_{kj} b_k^\dagger$, the observable

$$y(x) \equiv \langle n_m \rangle = \sum_j n_j |U_{mj}|^2 \quad (6.3)$$

for an initial occupation pattern $\{n_j\}$. Thus, by selecting which modes are initially populated by photons and which mode is measured, one realizes sums of mod-squared unitary entries determined by the circuit and the encoding.

A concrete, if wasteful, construction mirrors a classical weighted sum. Use $1 + 2n$ modes for an n -dimensional input x : one output mode o , data modes $d_1, \dots, d_n$, and paired ancilla modes $a_1, \dots, a_n$. Place one photon in each data mode and vacuum elsewhere. The encoding stage couples each pair (d_i, a_i) so that a prescribed fraction of the d_i amplitude is redirected to a_i . For example, set the amplitude to $\sqrt{x_i}$. The weight stage then couples each surviving d_i amplitude into the output o with coupling set by a learnable parameter w_i . This yields

$$y(x) \propto \sum_{i=1}^n \tilde{w}_i x_i \quad \text{with} \quad \tilde{w}_i = |w_i|^2,$$

up to a global normalization fixed by unitarity and the chosen beam-splitter conventions. An affine offset can be obtained by reserving a bias mode with a fixed photon and a calibrated coupling to o . This scheme consumes many modes and photons and offers no expressivity beyond a classical linear perceptron.

A more economical variant keeps the two-block structure but changes the roles. Use an encoding block $U_{\text{enc}}(x)$ to prepare task-aligned amplitudes, and let U_w be fully learnable subject to unitarity. For additional flexibility introduce a learnable pre-encoding unitary $U_{w'}$ so that

$$U(x; \theta) = U_w(\theta_w) U_{\text{enc}}(x) U_{w'}(\theta_{w'}), \quad (6.4)$$

which distributes input amplitudes across modes before encoding, enabling the circuit to form useful superpositions, balance amplitudes, and refine task-specific representations.

We take one designated output mode, n data modes for an n -dimensional input x , and $n_a \geq 1$ ancilla modes. The interferometer therefore acts on $m = n + 1 + n_a$ modes, so the unitaries are $m \times m$ with $m \geq n + 2$. To obtain nonclassical behavior with minimal resources, inject at least two indistinguishable photons into collision-free inputs, for example one in the output mode and one in an ancilla. Single-photon linear optics is efficiently classically simulable, whereas two or more photons produce genuinely nonclassical interference, including the Hong–Ou–Mandel effect. Unless stated otherwise, we use collision-free Fock inputs as in standard boson-sampling settings.

6.3 | MEASUREMENT-CONDITIONED QUANTUM ACTIVATION

We now specify a mechanism for a nonlinear, or more generally nonaffine, activation in the photon-native quantum perceptron. Uploading classical data is often nonlinear in parameter space and amounts to a projection within the relevant fixed-photon-number sector of the bosonic Fock space. Rather than rely on such encoding effects, we seek a state-dependent nonlinearity induced by measurement and feedforward; when the unit serves as a readout neuron we may additionally apply a classical nonlinearity in postprocessing.

After the weighted sum $U = U_w U_{\text{enc}}(x)$, we perform photon-number-resolving (PNR) measurements on all data modes and, optionally, on a subset of ancilla modes. At least one ancilla remains unmeasured; designate one such mode as the reference activation mode r . Each detector click pattern defines an outcome s in a finite set $\mathcal{S}$. A classical feedforward configurator associates to each s a two-mode unitary $U_{\text{ff}}(s)$ acting on the output mode o and the reference r . We implement $U_{\text{ff}}(s)$ by a tunable Mach–Zehnder interferometer with four learnable real parameters (two internal phases and two effective transmissivities). The mapping $s \mapsto \theta_s$ (MZI settings) is learned and stored as a dictionary.

Let M denote the measured modes and C the unmeasured block, with $\{o, r\} \subseteq C$. For the pre-measurement state

$$\rho(x) = U \rho_{\text{in}} U^\dagger, \quad (6.5)$$

and the PNR projector Π_s on M , the probability of outcome s and the conditional state on C are

$$p_s(x) = \text{Tr}[(\Pi_s \otimes I_C) \rho(x)], \quad \rho_{C|s}(x) = \frac{\text{Tr}_M[(\Pi_s \otimes I_C) \rho(x) (\Pi_s \otimes I_C)]}{p_s(x)}. \quad (6.6)$$

Applying the outcome-conditioned feedforward on $\{o, r\}$ yields

$$\rho_C^{\text{out}}(x; s) = [U_{\text{ff}}(s) \otimes I_{C \setminus \{o, r\}}] \rho_{C|s}(x) [U_{\text{ff}}(s)^\dagger \otimes I_{C \setminus \{o, r\}}]. \quad (6.7)$$

Each outcome s therefore selects a distinct mixing of o and r .

The activation induced by destructive PNR on M , followed by outcome-conditioned $U_{\text{ff}}(s)$ on $\{o, r\}$ and discarding all modes except o , is naturally a quantum *instrument* $\{\Phi_s\}_{s \in S}$ acting on the pre-measurement global state. For each click pattern s ,

$$\Phi_s(\rho_{\text{in}}) = \text{Tr}_{C \setminus \{o\}}[(U_{\text{ff}}(s) \otimes I) \text{Tr}_M[(\Pi_s \otimes I_C) U \rho_{\text{in}} U^\dagger] (U_{\text{ff}}(s)^\dagger \otimes I)], \quad (6.8)$$

is *completely positive and trace-non-increasing* (CPTNI), with $\text{Tr} \Phi_s(\rho_{\text{in}}) = p_s(x)$. Consequently:

- Per-branch behavior (per shot). In a single experimental run the next layer interacts with a concrete branch s . That branch is represented by the CPTNI instrument element Φ_s and yields a normalized conditional state on o for that run. It is trace-preserving (hence CPTP) *iff* $p_s(x) \equiv 1$ for all inputs, equivalently *iff* the POVM element E_s equals the identity, i.e., the trivial deterministic case.
- Non-selective (keep all outcomes). If a distinct $U_{\text{ff}}(s)$ is applied for every s and *no shots are discarded*, the ensemble map obtained by ignoring s across runs is

$$\Phi(\rho_{\text{in}}) = \sum_{s \in S} \Phi_s(\rho_{\text{in}}), \quad (6.9)$$

which is *completely positive and trace-preserving* (CPTP) because $\sum_s \Pi_s = I_M$. This CPTP map reproduces input-output statistics at the level of ensembles; it is not a per-shot state transformer.

- Selective post-selection. If only a subset $S \subset S$ is kept (e.g., discard runs where all photons were detected in M),

$$\Phi_S(\rho_{\text{in}}) = \sum_{s \in S} \Phi_s(\rho_{\text{in}}) \quad (6.10)$$

remains CPTNI with success probability $p_S(x) = \text{Tr} \Phi_S(\rho_{\text{in}}) \leq 1$.

For a non-input node we must still specify how to parameterize its encoder, even though there is no classical activation from the previous layer (the output mode o is typically not measured at non-terminal nodes). What we *do* observe per shot is the click pattern s on M , together with a deterministic, outcome-conditioned feed-forward $U_{\text{ff}}(s)$ from a lookup dictionary. We therefore define a shot-wise classical side signal by a map

$$(s, U_{\text{ff}}(s)) \mapsto a(s) \in [0, 1], \quad (6.11)$$

which is used by the next layer's encoder blocks while the quantum state on the unmeasured modes continues forward.

A convenient realization is

$$a(s) = \frac{N - N_M(s)}{N} T_{\text{oo}}(s) \quad (6.12)$$

where N is the number of photons injected into the perceptron and $N_M(s) = \sum_{i \in M} n_i(s)$ is the total detected on M for outcome s (the *survival fraction* is $(N - N_M)/N$). The factor $T_{\text{oo}}(s) \in [0, 1]$ is defined purely from the programmed two-mode block $U_{\text{ff}}(s)$ acting on the *ordered* pair (o, r) :

$$T_{\text{oo}}(s) = |\left[U_{\text{ff}}(s)\right]_{oo}|^2, \quad (6.13)$$

i.e., the intensity associated with the $o \rightarrow o$ intensity (mode o retention) of $U_{\text{ff}}(s)$. In a single-coupler parametrization with mixing angle $\theta(s)$ one has

$$T_{oo}(s) = \cos^2 \theta(s),$$

while an equivalent symmetric MZI convention with internal phase $\varphi(s)$ gives

$$T_{oo}(s) = \cos^2(\varphi(s)/2).$$

Either convention is acceptable; only the resulting $o \rightarrow o$ intensity matters in this scheme.

Intuitively, $(N - N_M)/N$ captures how much “photonic weight” remained unmeasured, while T_{oo} smoothly scales this quantity according to the node’s internal MZI setting. This construction yields a deterministic $a(s)$ without measuring o , suitable for classical fan-out. As a discrete alternative, one may apply a trainable hard threshold $\tau \in (0, 1)$ to the survival fraction:

$$a_{\text{hard}}(s) = \mathbf{1} \left[\frac{N - N_M(s)}{N} \geq \tau \right]. \quad (6.14)$$

If the perceptron node is terminal (readout), we perform PNR on *all* modes and return the per-shot click vector

$$\mathbf{n}^{(r)} = (n_1^{(r)}, \dots, n_m^{(r)}) \in \mathbb{N}_0^m \quad \text{for shot } r. \quad (6.15)$$

We allow two variants of the terminal readout:

- No conditioning (direct): set $U_{\text{ff}}(s) \equiv I$ and PNR all modes in one step. No side signal is produced.
- Outcome-conditioned (activated): first PNR the set M to obtain $s \in \mathcal{S}$, then apply the two-mode block $U_{\text{ff}}(s)$ on (o, r) , and finally PNR the remaining modes (so that, overall, all modes are measured). No side signal is produced.

The (population) *vector* readout is the infinite-shot limit

$$\mathbf{y}(x) \equiv \mathbb{E}[\mathbf{n} \mid x] = \sum_{\mathbf{n}} \mathbf{n} \Pr(\mathbf{n} \mid x) = \lim_{R \rightarrow \infty} \frac{1}{R} \sum_{r=1}^R \mathbf{n}^{(r)}, \quad (6.16)$$

with empirical estimator from R shots

$$\hat{\mathbf{y}}(x) = \frac{1}{R} \sum_{r=1}^R \mathbf{n}^{(r)}. \quad (6.17)$$

When a scalar summary is required (e.g., for a downstream classical stage), we use the output-mode component

$$y_o(x) = [\mathbf{y}(x)]_o = \mathbb{E}[n_o \mid x] = \lim_{R \rightarrow \infty} \frac{1}{R} \sum_{r=1}^R n_o^{(r)}, \quad \hat{y}_o(x) = \frac{1}{R} \sum_{r=1}^R n_o^{(r)}. \quad (6.18)$$

Different tasks admit different postprocessing. In the remainder of this report we take the *continuous* activation to be the population expectation $y_o(x)$, estimated empirically by $\hat{y}_o(x)$; thresholding or other classical transforms can be added when appropriate.

Algorithm 1 Photon-Native Quantum Perceptron

Require: Input sample x ; architecture $(M, C; o, r; N, n_a)$; outcome set $\mathcal{S}$; trainable parameters $\Theta = (\theta_{w'}, \theta_w, \{\theta_{\text{ff}}(s)\}_{s \in \mathcal{S}})$; encoder $U_{\text{enc}}(x)$; branch-selection subset $\mathcal{S}_*$ (empty $\Rightarrow$ non-selective); terminal readout mode $\text{rmode} \in \{\text{direct}, \text{activated}\}$; optional side-signal map $a : \mathcal{S} \rightarrow \mathbb{R}$ (non-terminal only)

Ensure: Terminal output: per-shot click vector $\mathbf{n} \in \mathbb{N}_0^m$ from PNR on all modes.

Ensure: Non-terminal output: if accepted (non-selective or $s \in \mathcal{S}_*$), forward quantum state on C , branch outcome s , optional $a(s)$; otherwise discard the run.

- 1: **Prepare** Fock input with total N photons (using n_a ancilla modes in C).
- 2: **Interferometer:** apply $U_{w'}(\theta_{w'}) \rightarrow U_{\text{enc}}(x) \rightarrow U_w(\theta_w)$.
- 3: **if** terminal node **then**
- 4: **if** $\text{rmode} = \text{direct}$ **then**
- 5: **Measurement:** PNR on $M \cup C$ to obtain the per-shot click vector $\mathbf{n} = (n_1, \dots, n_m) \in \mathbb{N}_0^m$.
- 6: **Activation:** skip (set $U_{\text{ff}}(s) \equiv I$). No side signal is produced.
- 7: **return** $\mathbf{n}$.
- 8: **else** $\triangleright \text{rmode} = \text{activated}$
- 9: **Measurement 1 on** M : PNR to obtain outcome $s \in \mathcal{S}$ and counts $\mathbf{n}_M$.
- 10: **Activation:** apply the outcome-conditioned two-mode block $U_{\text{ff}}(s)$ on the ordered pair (o, r) .
- 11: **Measurement 2 on** C : PNR to obtain counts $\mathbf{n}_C$.
- 12: **Assemble:** form $\mathbf{n}$ over all modes (in fixed mode order) from $(\mathbf{n}_M, \mathbf{n}_C)$.
- 13: No side signal is produced.
- 14: **return** $\mathbf{n}$.
- 15: **end if**
- 16: **else** $\triangleright$ non-terminal node
- 17: **Measurement on** M : PNR to obtain outcome $s \in \mathcal{S}$ and the corresponding conditional state on C .
- 18: **if** $\mathcal{S}_* = \emptyset$ **or** $s \in \mathcal{S}_*$ **then** $\triangleright$ non-selective or accepted branch
- 19: **Activation:** apply the outcome-conditioned two-mode block $U_{\text{ff}}(s)$ on the ordered pair (o, r) .
- 20: **Optional side signal:** compute $a(s)$ if defined.
- 21: **return** (quantum state on C , branch outcome s , optional $a(s)$) to the next layer.
- 22: **else** $\triangleright$ rejected by selective branch selection
- 23: Discard the current run.
- 24: **end if**
- 25: **end if**

6.4 | A MINIMAL ARCHITECTURE

We fix n data modes and output o and reference r ordered as $(o, r, d_1, \dots, d_n)$. Each forward pass injects the collision-free Fock state $|1_o, 1_r, 0, \dots, 0\rangle$. The encoder acts trivially on o and unitarily on the $(r, \mathbf{d})$ subspace.

Householder encoder. Let $|e_1\rangle = (1, 0, \dots, 0)^\top$ on the $(r, \mathbf{d})$ block and choose the target

$$|v_{\text{enc}}(x)\rangle = (\alpha, \chi_1, \dots, \chi_n)^\top, \quad \alpha = \sqrt{1 - \frac{\|x\|_1}{n}}, \quad \chi_j = \sqrt{\frac{x_j}{n}} e^{2\pi i x_j}, \quad (6.19)$$

so $\| |v_{\text{enc}}(x)\rangle \| = 1$ for $x \in [0, 1]^n$ (or after rescaling to ensure $|x|_1 \leq n$). Define the Householder vector

$$v = \frac{|e_1\rangle - |v_{\text{enc}}(x)\rangle}{\| |e_1\rangle - |v_{\text{enc}}(x)\rangle \|} = \frac{(1 - \alpha, -\chi_1, \dots, -\chi_n)^\top}{\sqrt{2(1 - \alpha)}}, \quad (6.20)$$

and the reflector $B_\chi(x) = I - 2 v v^\dagger$, which satisfies $B_\chi(x) |e_1\rangle = |v_{\text{enc}}(x)\rangle$. The encoder is block-diagonal

$$U_x \equiv U_{\text{enc}}(x) = \begin{pmatrix} 1 & 0 \\ 0 & B_\chi(x) \end{pmatrix} \in U(n+2), \quad (6.21)$$

with $U_x = I_{n+2}$ when $x = 0$ (so $\alpha = 1, \chi = 0$ and $|v_{\text{enc}}\rangle = |e_1\rangle$). For $x \neq 0$, the explicit entries on the $(\mathbf{d}, \mathbf{d})$ block ($1 \leq i, j \leq n$) are

$$(U_x)_{ij} = \delta_{ij} - \frac{\chi_i \chi_j^*}{1 - \alpha}, \quad (6.22)$$

and $(U_x)_{r,r} = \alpha, (U_x)_{r,d_j} = \chi_j^*, (U_x)_{d_j,r} = \chi_j$.

The global unitary is the $(n+2) \times (n+2)$ block:

$$U_x = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & \alpha & \chi_1^* & \cdots & \chi_n^* \\ 0 & \chi_1 & 1 - \frac{|\chi_1|^2}{1 - \alpha} & \cdots & -\frac{\chi_1 \chi_n^*}{1 - \alpha} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \chi_n & -\frac{\chi_n \chi_1^*}{1 - \alpha} & \cdots & 1 - \frac{|\chi_n|^2}{1 - \alpha} \end{pmatrix}. \quad (6.23)$$

Trainable parameters. We retain the block layout

$$U(\theta; x) = U_w(\theta_w) U_x U_{w'}(\theta_{w'}) \in U(n+2),$$

and define the full parameter set

$$\Theta = (\theta_{w'}, \theta_w, \{\theta_{\text{ff}}(s)\}_{s \in \mathcal{S}}). \quad (6.24)$$

Activation and readout. *Non-terminal:* use the non-selective activation; no side signal. *Terminal:* across shots, aggregate to estimate the output-mode expectation $\hat{y}_o(x) = \frac{1}{R} \sum_{r=1}^R n_o^{(r)}$.

Otherwise as previously defined.

For the remainder of this report, we adopt the minimal PnQP architecture that preserves all essential behavior.

6.5 | SIMULATIONS AND TRAINING

We evaluate a *single* PnQP node used as an *activated terminal readout* (cf. Algorithm 1) on small 2D binary benchmarks and compare against lightweight classical baselines under identical splits.

Data. We use three instances each of the *moons* (varying noise), *circles* (varying separation and noise), and *linear* (varying class separation) datasets, and one instance of the *XOR* dataset. Each dataset has 200 data points with a fixed 70/30 train/test split. Features are min-max scaled to $[0, 1]$ to match the encoder domain. Except for the linear case, the datasets are not linearly separable and therefore provide informative cases in which a classical perceptron with a single linear decision boundary fails.

Model. We use the minimal architecture introduced above. The interferometer is the composed unitary $U = U_w U_{\text{enc}}(x) U_{w'}$. We parameterize only the *first two rows* of U_w and the *first two columns* of $U_{w'}$ and complete them to full unitaries via a QR retraction after each update (compact parameterization with guaranteed unitarity). At the terminal node, we apply the outcome-conditioned 2×2 block to (o, r) and then perform PNR on all modes. The continuous learning signal is the empirical output-mode population $\hat{y}_o$, and a learned threshold τ binarizes predictions.

Optimization. We optimize $(\theta_w, \theta_{w'}, \{\theta_{\text{ff}}(s)\}, \tau, \log s)$ using Simultaneous Perturbation Stochastic Approximation (SPSA) [112, 113]. Each update uses two forward evaluations of the objective (or $2m$ with m -pair averaging), with random $\pm$ perturbations to estimate a descent direction. The loss is a logistic margin on $\hat{y}_o$ centered at τ with temperature $s = \exp(\log s)$. Step sizes are auto-calibrated, periodic parameters are wrapped to their natural ranges, and the QR retraction enforces unitarity.

Protocol and ablations. We ramp shot counts during training and report both *ideal* and *noisy* simulations. The noise model includes partial indistinguishability and second-order correlation at the photon source, with representative parameters guided by Quandela device characterizations (Ascella [114] and Belenos[115]). As an ablation, we also test removing $U_{w'}$ while keeping all other settings fixed.

Implementation. All photonic simulations are performed in Perceval [116], an open-source, photonics-native Python framework from Quandela, using the strong-simulation backend SLOS and the library’s feed-forward configurator to realize the outcome-conditioned 2×2 block.

Results. Across all datasets (Figure 6.1), the PnQP identifies high-quality decision boundaries and consistently solves XOR. On *moons*, *circles*, and *linear* benchmarks, its performance is competitive with classical baselines. Among the classical models (excluding the MLP and the vanilla perceptron), accuracies are generally similar, with the SVM achieving the highest average accuracy (92%). For comparability, all classical models—apart from the perceptron baseline, were configured to have a number of trainable parameters comparable to the PnQP.

The classical perceptron performs well only on the *linear* dataset; on the nonlinearly separable tasks it is constrained by its single linear decision boundary and typically remains only modestly above chance (50%), with occasional improvements due to noise and class asymmetries.

The ideal (noise-free) PnQP attains 91% accuracy, essentially matching the strongest classical baselines. Notably, the Belenos-characterized noisy simulation (partial indistinguishability = 0.651, $g^{(2)} = 0.03$) reaches a similar accuracy, whereas the Ascella-characterized setting (partial indistinguishability = 0.8722, $g^{(2)} = 0.016$) yields a more pronounced degradation. An ablated variant retaining only the encoder and the second weight unitary achieves 90.2% accuracy, supporting the hypothesis that the Householder-based encoder provides a beneficial inductive bias by mixing modes prior to learning.

Qualitatively, the PnQP’s activation surfaces are expressive on the two-dimensional benchmarks, enabling a broad family of decision boundaries. While it is unclear whether this family is universal, practical limitations from training and finite-shot estimation will in any case constrain the set of realizable boundaries. See Figure 6.2 for the activation surface and decision boundary for the XOR. See decision boundaries for circles (Figure 6.3) and activation surfaces for the moons (Figure 6.4) below.

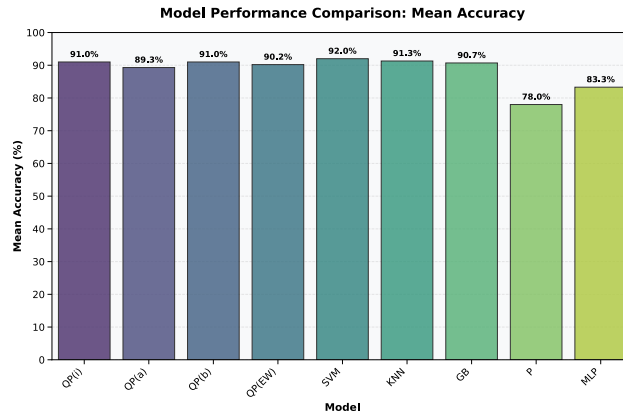


Figure 6.1: Mean accuracy comparison for the Quantum Perceptrons (QP) with (i) ideal, (a) ascella, (b), belenos, (EW) ablation without first trainable block, SVM, KNN, Gradient Boosting (GB), classical perceptron (P), MLP

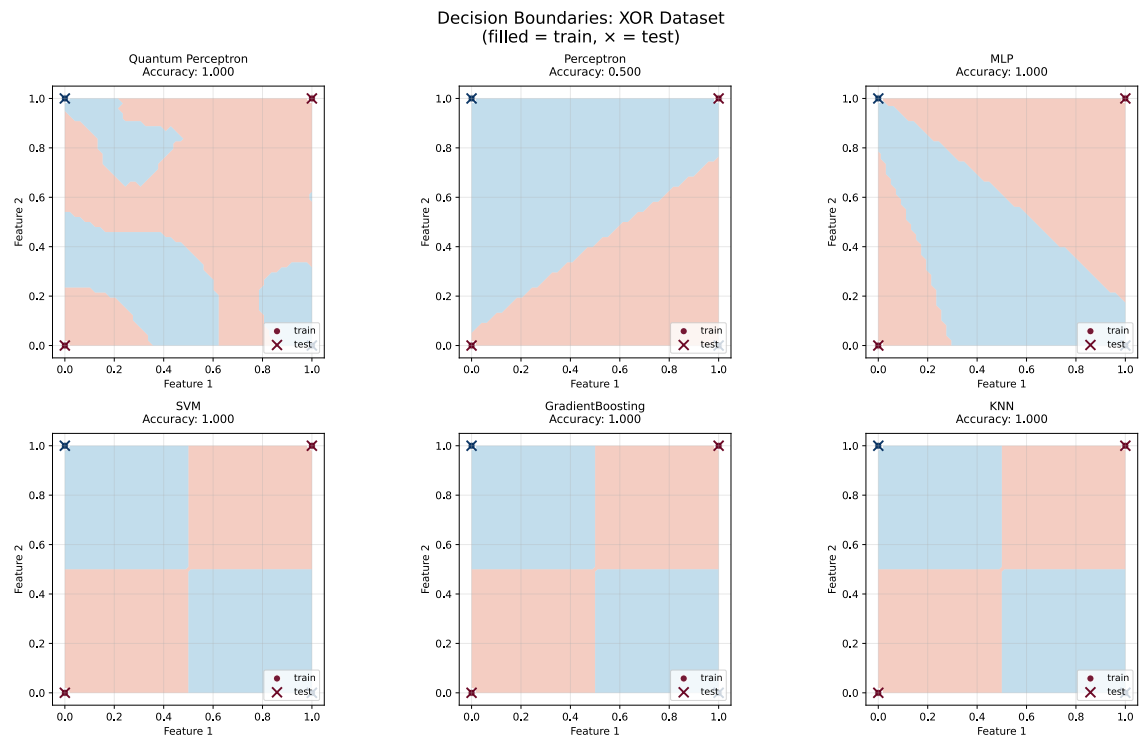
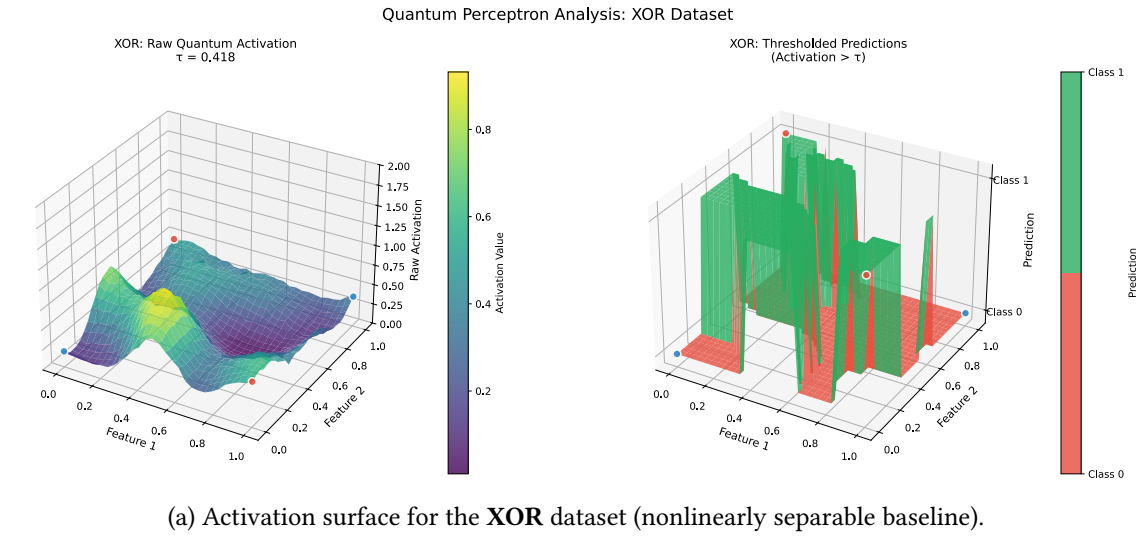
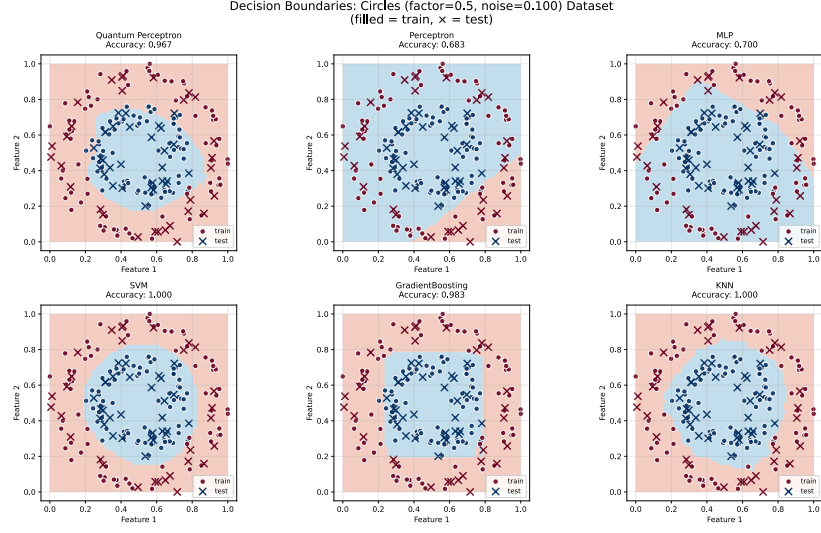
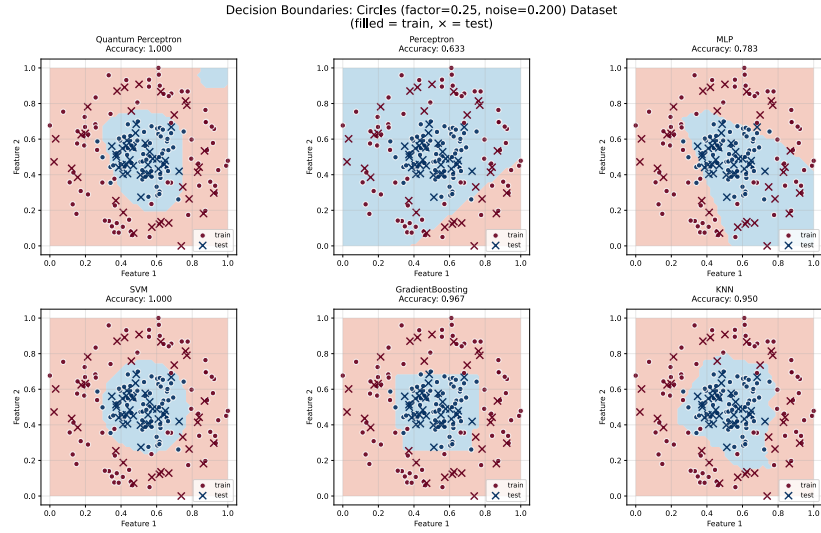


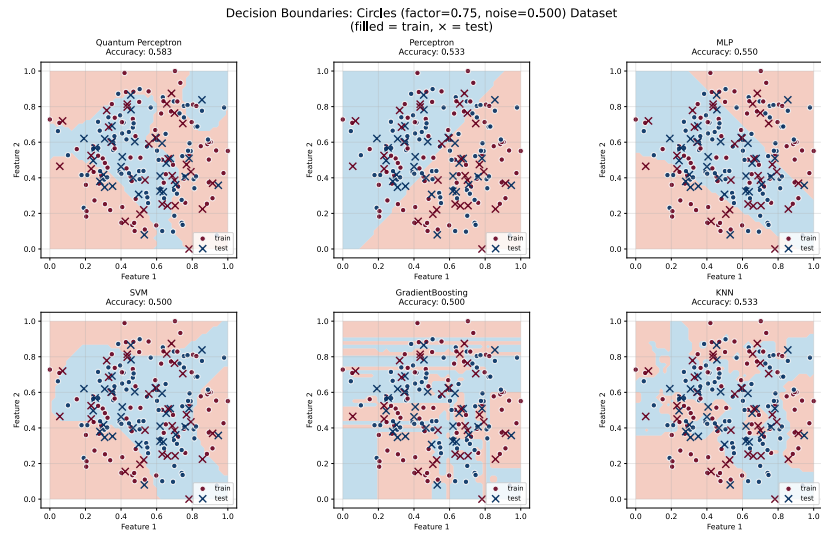
Figure 6.2: Results for the XOR dataset.



(a) factor = 0.50, noise = 0.10



(b) factor = 0.25, noise = 0.20



(c) factor = 0.75, noise = 0.50

Figure 6.3: Decision boundaries for the Circles dataset under varying ring factor and label noise.

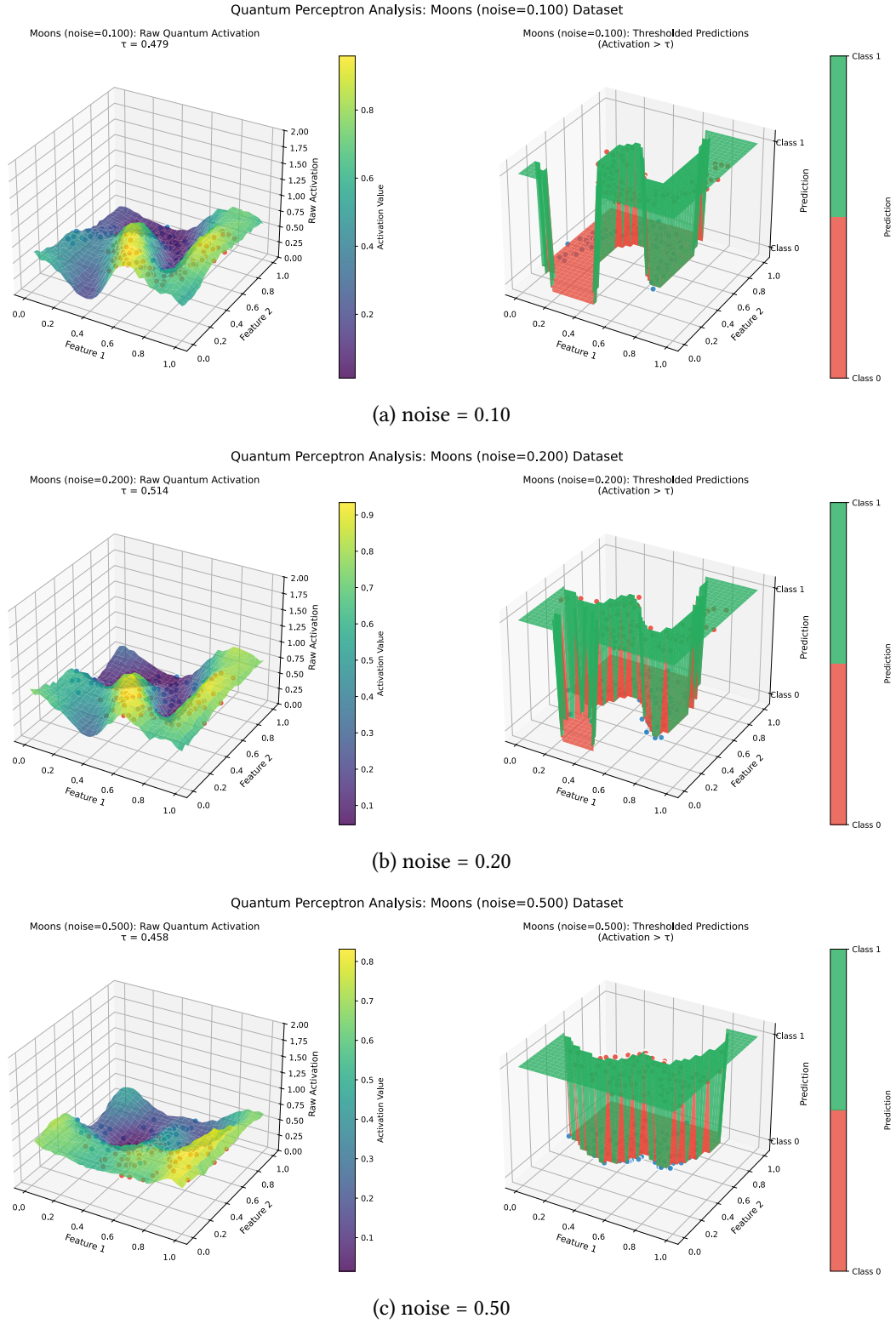


Figure 6.4: Activation surfaces for the Moons dataset as label noise increases.

PHOTON-NATIVE QUANTUM NEURAL NETWORK

A PnQP (Algorithm 1) could, in principle, forward unmeasured modes to subsequent nodes, thereby preserving inter-node quantum correlations. In practice, however, simulating such multi-node states is quickly intractable: the simulator must carry the full joint state across nodes and, with mid-circuit feed-forward, condition next-layer parameters on classical outcomes $a(s)$ and (re)compile the updated circuit on the fly. Physically, the light must be stored in an optical delay line while controls settle.

Even the single-node PnQPs used in the previous sections are already resource-hungry; composing many such nodes into a network is prohibitive and leads to out-of-memory failures on modest hardware. To keep inference tractable, we adopt a *terminal-readout* policy specified at the layer level: each node is executed in terminal mode, and all modes are measured at the end of every shot. This decouples nodes within a layer, allowing us to simulate the perceptrons independently and to pass only classical summaries to the next layer.

In the sections that follow, we will build on this idea; however, it should be noted that these neural networks are better characterized as essentially photon-native quantum perceptron networks with classical communication. The essential framework is shown in Figure 7.1. For brevity, we will continue to refer to them in simpler terms, with no intent to mislead.

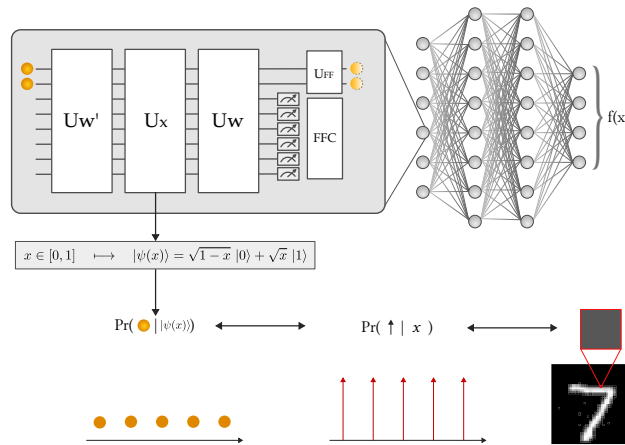


Figure 7.1: Photon-native quantum neural network with classical communication. Each node is a photon-native quantum perceptron. The spatial modes are measured and shot-averaged to yield mean photon counts as outputs. The photon-native quantum perceptron unit can be interpreted as a spiking unit when the number of shots is low; it encodes information in its amplitudes, paralleling how spiking neurons encode information in the probability of spiking.

7.1 | PHOTON-NATIVE PERCEPTRON LAYER

Layers are indexed by $\ell = 1, \dots, L$ with widths $(n_0 = d, n_1, \dots, n_L)$. Let the input to layer ℓ be $\mathbf{z}^{(\ell-1)}$ ($\mathbf{z}^{(0)} = \mathbf{x} \in [0, 1]^{n_0}$; thereafter $\mathbf{z}^{(\ell-1)}$ is the postprocessed output of the previous layer). Node (ℓ, k) carries parameters $\Theta^{(\ell, k)}$.

All nodes run Algorithm 1 in *terminal* mode with *direct* readout ($U_{\text{ff}} \equiv I$), keep all outcomes ($S_* = \emptyset$), use two photons ($N_\ell = 2$) with collision-free injection, and PNR all modes per shot. For node (ℓ, k) , shot r returns a click vector $\mathbf{n}$; we record $n_o^{(\ell, k; r)} = [\mathbf{n}]_o$.

A photon-native perceptron layer (PnQPL) comprises n_ℓ independent nodes acting on the common input $\mathbf{z}^{(\ell-1)}$; each performs R_ℓ shots and returns the shot-average output-mode estimate as its activation:

$$\hat{y}_o^{(\ell, k)} = \frac{1}{R_\ell} \sum_{r=1}^{R_\ell} n_o^{(\ell, k; r)}, \quad \hat{\mathbf{y}}_o^{(\ell)} = (\hat{y}_o^{(\ell, 1)}, \dots, \hat{y}_o^{(\ell, n_\ell)})^\top.$$

An optional elementwise postprocessing σ (identity, clipping, logistic) may be applied: $\hat{\mathbf{y}}_o^{(\ell)} \leftarrow \sigma(\hat{\mathbf{y}}_o^{(\ell)})$. We denote the postprocessed layer output by $\mathbf{z}^{(\ell)}$.

Algorithm 2 Photon-native Quantum Perceptron Layer

Require: Input $\mathbf{z}^{(\ell-1)} \in [0, 1]^{n_{\ell-1}}$; width n_ℓ ; shots R_ℓ ; node params $\{\Theta^{(\ell, k)}\}_{k=1}^{n_\ell}$; optional σ

Ensure: $\hat{\mathbf{y}}_o^{(\ell)} \in \mathbb{R}^{n_\ell}$ and (if σ given) $\mathbf{z}^{(\ell)} \in \mathbb{R}^{n_\ell}$

```

1: for  $k = 1$  to  $n_\ell$  do                                     ▷ parallel when hardware permits
2:   for  $r = 1$  to  $R_\ell$  do
3:      $\mathbf{n} \leftarrow \text{PNQPFORWARD}(\mathbf{z}^{(\ell-1)}; \Theta^{(\ell, k)})$       ▷ Algorithm 1, terminal, direct
4:      $n_o^{(\ell, k; r)} \leftarrow [\mathbf{n}]_o$ 
5:   end for
6:    $\hat{y}_o^{(\ell, k)} \leftarrow \frac{1}{R_\ell} \sum_{r=1}^{R_\ell} n_o^{(\ell, k; r)}$ 
7: end for
8:  $\hat{\mathbf{y}}_o^{(\ell)} \leftarrow (\hat{y}_o^{(\ell, 1)}, \dots, \hat{y}_o^{(\ell, n_\ell)})^\top$ 
9: if  $\sigma$  is provided then
10:   $\mathbf{z}^{(\ell)} \leftarrow \sigma(\hat{\mathbf{y}}_o^{(\ell)})$ 
11: else
12:   $\mathbf{z}^{(\ell)} \leftarrow \hat{\mathbf{y}}_o^{(\ell)}$ 
13: end if
14: return  $\hat{\mathbf{y}}_o^{(\ell)}$  and  $\mathbf{z}^{(\ell)}$ 

```

7.2 | PHOTON-NATIVE QNN

A PnQNN composes PnQPLs:

$$f_\Theta : \mathbf{x} \in [0, 1]^{n_0} \mapsto \hat{\mathbf{y}}_o^{(L)} \in \mathbb{R}^{n_L}, \quad \hat{\mathbf{y}}_o^{(\ell)} = \text{PnQPL}(\mathbf{z}^{(\ell-1)}; \Theta^{(\ell)}, R_\ell),$$

with $\Theta^{(\ell)} = \{\Theta^{(\ell, k)}\}_{k=1}^{n_\ell}$ and $\mathbf{z}^{(0)} = \mathbf{x}$, $\mathbf{z}^{(\ell)} = \sigma_\ell(\hat{\mathbf{y}}_o^{(\ell)})$ (identity if no postprocessing). A task head (e.g., softmax) may be applied to $\mathbf{z}^{(L)}$.

In this regime, each node behaves as a classical stochastic channel. The encoding itself induces nonlinearity by mapping the input into a quantum state through a generally nonlinear feature map in parameter space. Moreover, multi-photon interference renders the PNR outcome probabilities nonlinear functions of the input and circuit parameters, so the mean output count is nonlinear as well. Optional classical postprocessing between layers may further shape this effect.

Nodes support *distributed execution* across photonic processors, or *fully serialized execution* on a single device (one node at a time, layer by layer).

Algorithm 3 Photon-native Quantum Neural Network

Require: $\mathbf{x} \in [0, 1]^{n_0}$; widths $(n_0, \dots, n_L)$; shots $\{R_\ell\}$; params $\Theta = \{\Theta^{(\ell)}\}_{\ell=1}^L$; optional $\{\sigma_\ell\}$; task head ψ

Ensure: $\hat{\mathbf{y}} \in \mathbb{R}^{n_L}$

```

1:  $\mathbf{z}^{(0)} \leftarrow \mathbf{x}$ 
2: for  $\ell = 1$  to  $L$  do
3:    $\hat{\mathbf{y}}_o^{(\ell)} \leftarrow \text{PNQPL}(\mathbf{z}^{(\ell-1)}; \Theta^{(\ell)}, R_\ell, \sigma_\ell)$ 
4:    $\mathbf{z}^{(\ell)} \leftarrow \begin{cases} \sigma_\ell(\hat{\mathbf{y}}_o^{(\ell)}), & \text{if } \sigma_\ell \text{ provided} \\ \hat{\mathbf{y}}_o^{(\ell)}, & \text{otherwise} \end{cases}$ 
5: end for
6:  $\hat{\mathbf{y}} \leftarrow \psi(\mathbf{z}^{(L)})$ 
7: return  $\hat{\mathbf{y}}$ 

```

7.3 | IMAGE CLASSIFICATION

We study whether a *general-purpose* photon-native quantum neural network (PnQNN) can function as a drop-in alternative to a multilayer perceptron (MLP) under controlled conditions. To attribute performance differences to the learning mechanism rather than to feature engineering, we deliberately exclude specialized inductive biases (e.g., convolutions, attention) and train dense photon-native layers on compact, *untrained* features. The training protocol and input features are matched across models.

Implementation. With a single photon injected in each of input modes 1 and 2 and terminal photon counting at the output, the expected count at any output mode equals the sum of two squared complex overlaps. Concretely: select the target output row of the right unitary U_w , propagate it through the input-dependent encoding unitary, and compute overlaps with the first two columns of the left unitary U'_w . Because only first-order moments contribute, interference cross-terms vanish. The resulting expression is analytic and vectorizable.

All overlaps for all outputs and minibatch items are computed with GPU-accelerated matrix multiplications and elementwise operations. The computation is differentiable end-to-end and compatible with automatic differentiation.

Datasets

- **MNIST.**¹ 70,000 grayscale 28×28 images (60,000 train / 10,000 test) labeled 0–9. A *fixed* random linear projection maps $\mathbb{R}^{784} \rightarrow \mathbb{R}^{64}$ and is shared by all models. For visualization only, the 64-D vector is reshaped to 8×8 (Fig. 7.2a); both panels are summarized in Fig. 7.2.
- **CIFAR-10.**² 60,000 color 32×32 images (50,000 train / 10,000 test) across 10 object categories. We apply the same *random* projection $\mathbb{R}^{3072} \rightarrow \mathbb{R}^{64}$ and visualize as 8×8 (Fig. 7.2b); both datasets are summarized in Fig. 7.2.

Experimental Protocol

Training procedure. Unless stated otherwise, training runs for **100 epochs** with Adam (initial learning rate 10^{-3} , monotonically decayed), identical batching, and identical train/test splits. The MLP employs batch-normalized linear layers with ReLU activations; the PnQNN uses its native photon-derived nonlinearity.

¹<http://yann.lecun.com/exdb/mnist/>

²<https://www.cs.toronto.edu/~kriz/cifar.html>



Figure 7.2: Visualization of the reduced 64-D features for both datasets (reshaping is for display only).

Comparison design. Photon-native quantum perceptrons have substantially more tunable parameters *per unit* than MLP neurons, but *both* architectures emit a single scalar per unit. Consequently, for a layer with width m that maps $\mathbb{R}^d \rightarrow \mathbb{R}^m$, the dimensionality transform from d to m is identical. What differs is the *resource per activation*: parameters required to produce each scalar output can vary by orders of magnitude. To make comparisons interpretable, we evaluate two regimes on *both* datasets:

1. **Network size-matched:** fix depth and widths identically for the MLP and the PnQNN with the same $(n_0, \dots, n_L)$, thereby matching the exported representation dimensions layer by layer.
2. **Approximate parameter matching:** shrink the PnQNN and enlarge the MLP so that total parameters are comparable.

Metrics. We report accuracy, wall-clock training time, and calibration metrics: *Expected Calibration Error* (ECE; weighted average of bin-wise confidence-accuracy gaps), *Maximum Calibration Error* (MCE; worst-bin gap), and the *Brier score* (mean squared error of predicted class probabilities). Reliability diagrams accompany numerical summaries.

MNIST

NETWORK SIZE-MATCHED.

64→32→32→32→10. PnQNN: 450,174 parameters. MLP: 4,714 parameters.

Results. The PnQNN reaches ~90% test accuracy earlier and stabilizes at a higher level than the compact MLP. Reliability and learning dynamics are shown in Fig. 7.3a and Fig. 7.3b; both are summarized in Fig. 7.3. Results indicate broadly similar error modes, with fewer high-confidence errors for the PnQNN.

APPROXIMATE PARAMETER MATCHING.

PnQNN: 118,054 parameters (64→12→12→12→10). MLP: 152,330 parameters (64→256→256→256→10).

Results. The MLP is ahead on accuracy and speed: final test accuracy 95.5% (best $\approx 95.6\%$) vs. PnQNN 90.9% (best $\approx 91.1\%$); the MLP reaches 90% after 1 epoch, whereas the PnQNN requires 52 epochs. Late-epoch stability (standard deviation over the last 10 epochs) favors the MLP (0.045 vs. 0.190). Calibration is mixed: the PnQNN exhibits lower *ECE* (0.0859 vs. 0.1035) and *MCE* (0.1695 vs. 0.2092), but the MLP’s higher accuracy yields a better *Brier* score (0.0837 vs. 0.1441). These outcomes are visualized in Fig. 7.4a and Fig. 7.4b, summarized in Fig. 7.4.

CIFAR-10

NETWORK SIZE-MATCHED.

64→32→32→32→10. PnQNN: 450,174 parameters. MLP: 4,714 parameters.

Results. The PnQNN modestly exceeds the compact MLP in top-1 accuracy (40.19% vs. 38.82%). Calibration in this regime favors the MLP (*ECE* 0.0118, *MCE* 0.0249, *Brier* 0.7460) over the PnQNN (*ECE* 0.0600, *MCE* 0.1031, *Brier* 0.7522). On the same hardware, wall-clock training times are 3187.3 s for the PnQNN and 148.3 s for the MLP. Figure 7.5a reports calibration details; accuracy dynamics and parameter/time summaries appear in Fig. 7.5b.

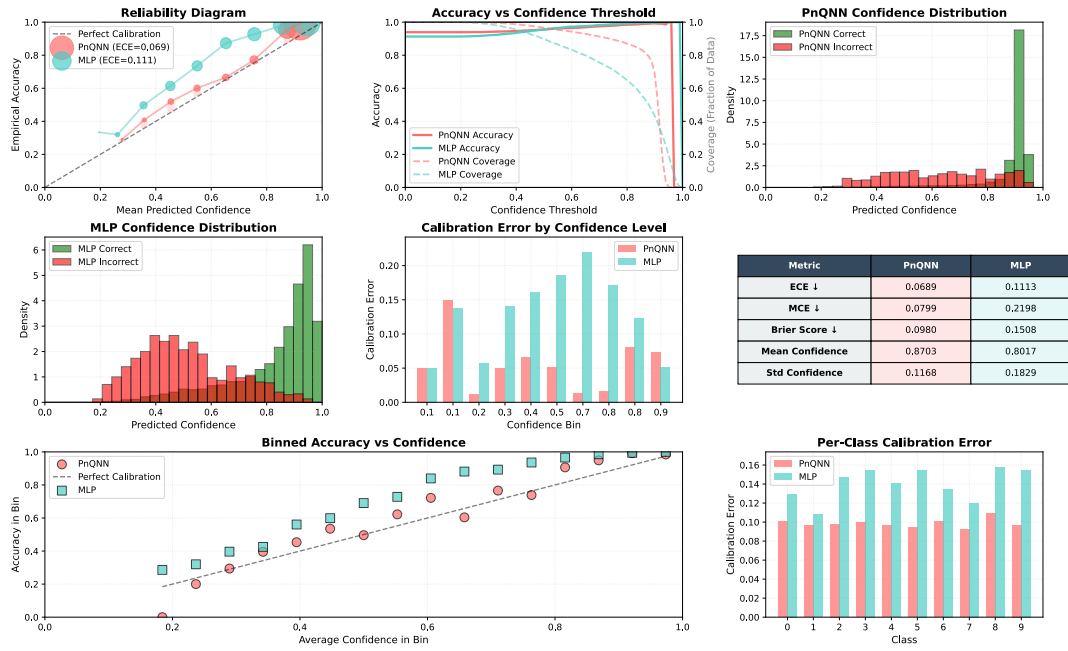
APPROXIMATE PARAMETER MATCHING.

PnQNN: 118,054 parameters (64→12 custom→10). MLP: 152,330 parameters (64→256→256→256→10).

Results. The PnQNN trains slower (1563.3 s vs. 319.7 s). Accuracy slightly favors the MLP: best 39.56% vs. PnQNN 38.36%; the MLP peaks early (epoch 3), whereas the PnQNN converges slowly and plateaus near 38.4% by epoch 100. Late-epoch stability (standard deviation over the last 10 epochs) favors the MLP (0.119 vs. 0.169). Under matched budgets, calibration reverses: the PnQNN exhibits markedly lower *ECE* (0.0127) and *MCE* (0.0966), and a better *Brier* (0.7476) than the MLP (*ECE* 0.3003, *MCE* 0.4302, *Brier* 0.9414). The MLP also overfits (reaching 98.62%) on the training data but generalizes poorly. Fig. 7.6 displays the results.

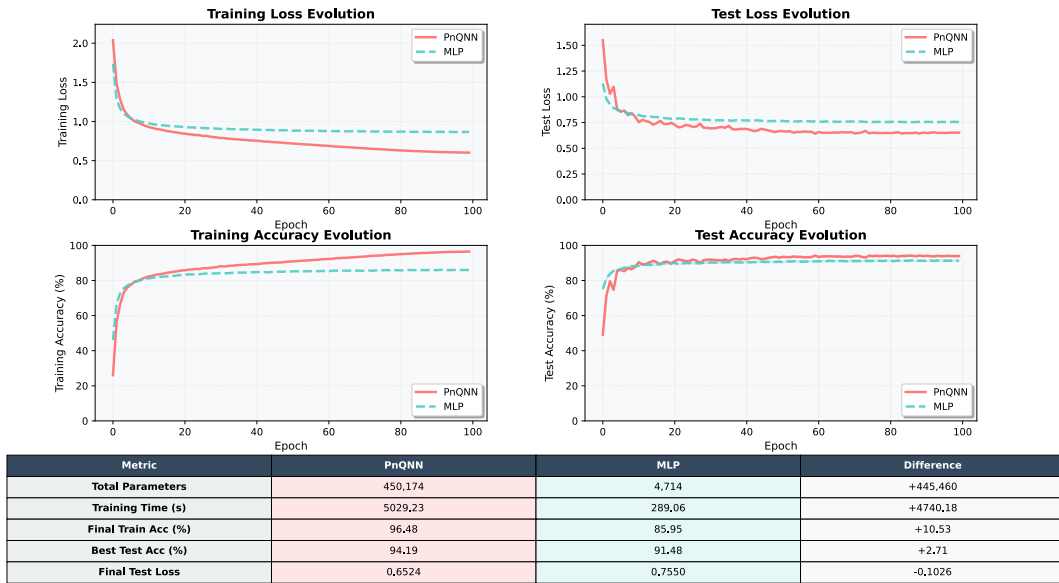
Discussion The PnQNN, evaluated in the terminal-readout surrogate with shared 64-D features, is competitively close to compact MLPs but exhibits a clear accuracy–calibration–throughput trade: in the size-matched setting (64→32→32→32→10) it learns faster on MNIST and modestly exceeds the compact MLP on CIFAR-10, albeit with weaker calibration; under approximate parameter matching (narrow PnQNN vs. wide MLP), the MLP decisively leads in accuracy and speed on both datasets (MNIST $\approx 95.5\%$ vs. 90.9%; CIFAR-10 best $\approx 39.6\%$ vs. 38.4%), while the PnQNN often delivers better uncertainty estimates (lower *ECE*/*MCE*—and on CIFAR-10 also a better *Brier*—despite lower accuracy). These outcomes align with the one-scalar-per-unit constraint: added per-unit expressivity in photon-native layers improves calibration and early learning but cannot substitute for missing width when upstream features are compact, whereas widening the MLP increases feature rank and peak accuracy. Thus, in this regime the practical advantage of PnQNNs is calibrated probabilities under tight parameter budgets, with headroom for accuracy and efficiency via multi-output photon-native blocks and tighter/factorized unitary parameterizations.

Model Calibration & Confidence Analysis



(a) Reliability diagram and calibration metrics.

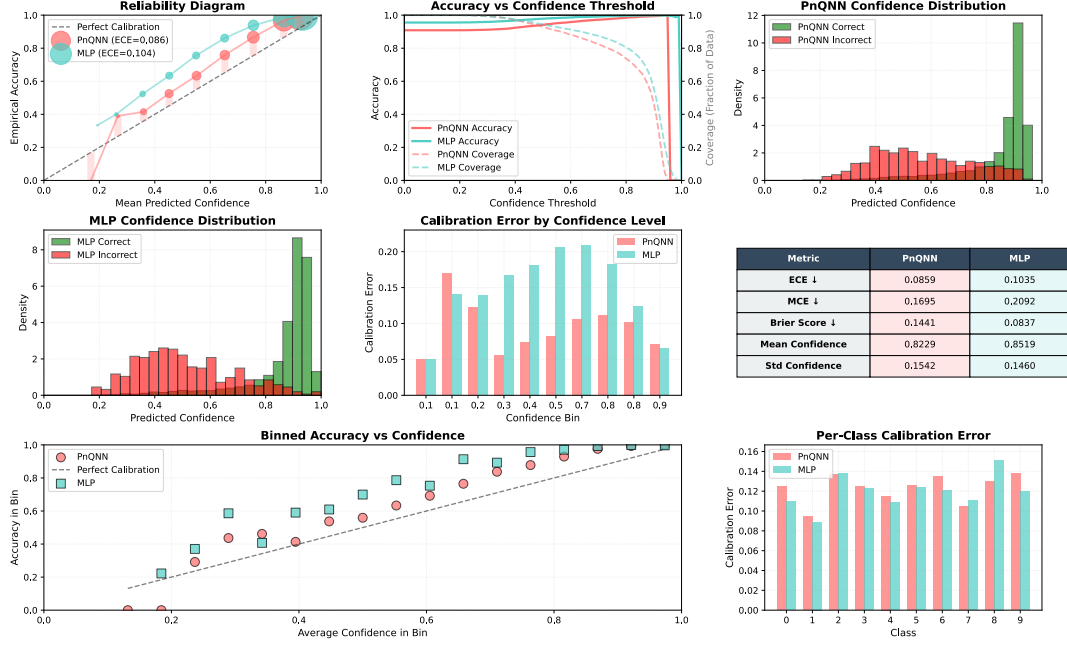
Model Performance Comparison: PnQNN vs MLP



(b) Test accuracy vs. epoch.

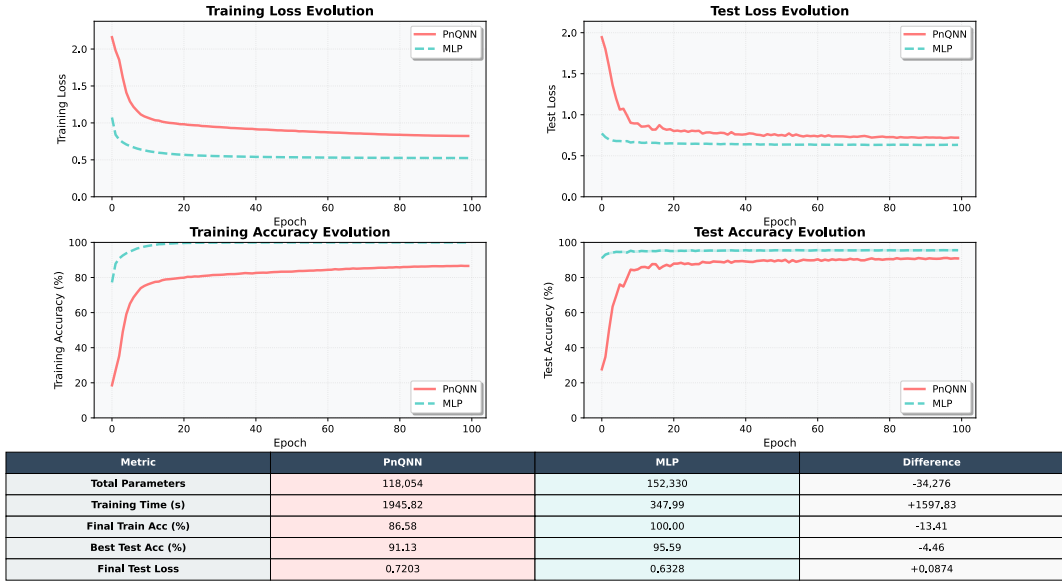
Figure 7.3: Reduced MNIST Benchmark (PnQNN: 450k; MLP: 4.7k).

Model Calibration & Confidence Analysis



(a) Reliability diagram and calibration metrics

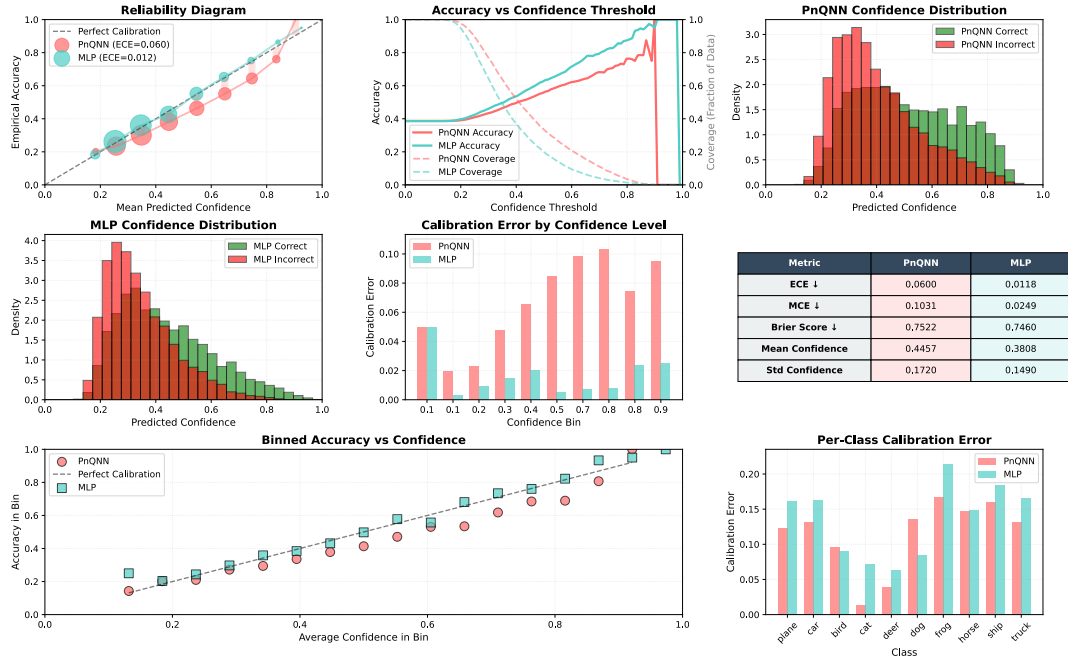
Model Performance Comparison: PnQNN vs MLP



(b) Test accuracy vs. epoch.

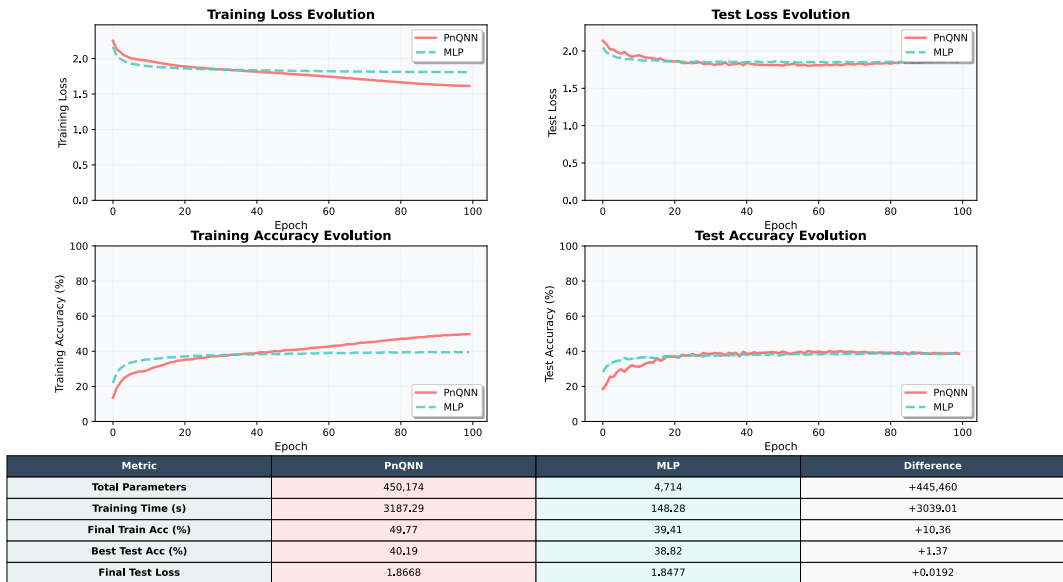
Figure 7.4: Reduced MNIST Benchmark (PnQNN: 118k; MLP: 152k).

Model Calibration & Confidence Analysis



(a) Reliability diagram and calibration metrics.

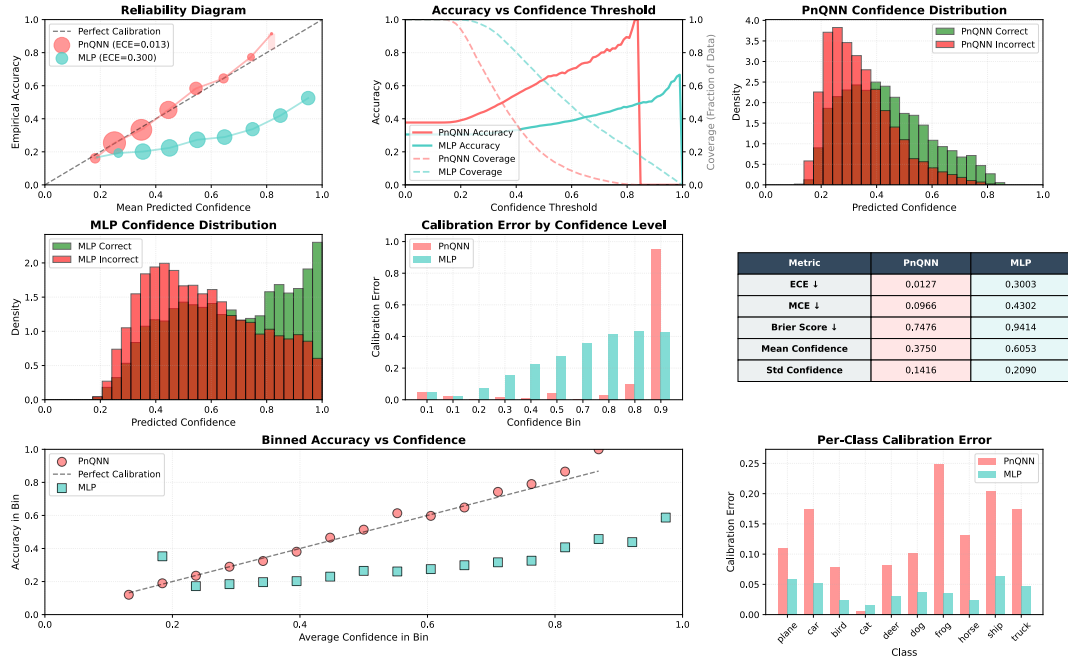
Model Performance Comparison: PnQNN vs MLP



(b) Test accuracy vs. epoch.

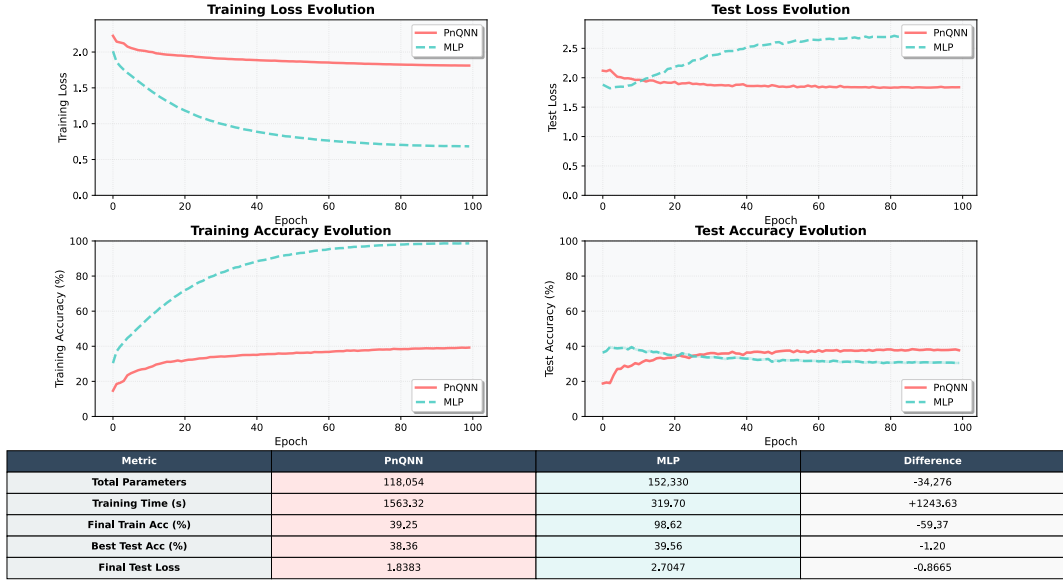
Figure 7.5: Reduced CIFAR-10 Benchmark (PnQNN: 450k; MLP: 4.7k).

Model Calibration & Confidence Analysis (CIFAR-10)



(a) Reliability diagram and calibration metrics.

Model Performance Comparison: PnQNN vs MLP (CIFAR-10)



(b) Test accuracy vs. epoch.

Figure 7.6: Reduced CIFAR-10 Benchmark (PnQNN: 118k; MLP: 152k).

CONCLUSION

This thesis specifies a photon-native route to neural computation. Neurons are realized with linear optics. Measurement, optionally followed by feed-forward, supplies the nonlinearity. A photon-native quantum perceptron (PnQP) is defined and composed into simulator-ready layers (PnQNNs) that exchange classical outcomes.

A single two-photon PnQP with outcome-conditioned feed-forward learns nonlinear decision boundaries. Encoders that mix mode amplitudes, notably Householder mixers motivated by spiking-neuron intuition, provide an inductive bias. On tasks with compact, shared features, photon-native layers achieve accuracies comparable to MLPs and exhibit tighter calibration, which is material when confidence estimates must be trusted. Representational capacity is constrained primarily by width rather than by per-unit parameter count. Each unit exposes one scalar; additional internal expressivity does not compensate for missing channels.

All reported networks were trained with a conservative classical surrogate. The pipeline did not preserve quantum state across nodes, apply selective post-selection, or construct deeper feed-forward trees. The resulting expressivity claims are strict lower bounds relative to fully photon-native execution, and stronger network effects are expected in the full model. The surrogate remains valuable because it enables classical pretraining prior to hardware realization.

Future work targets optimization and full hardware realizability: maintain inter-node quantum state, introduce selective post-selection, and add multi-stage feed-forward to align the model with feasible photonic hardware. Increase feature rank while reducing parameter cost through multi-output photon-native blocks, factorized or tied meshes, and more efficient encoders. Additional promising directions briefly studied in this thesis include photonic reservoir computing, photonic memristors for recurrence-like behaviour, and quantum neural tangent kernels for diagnostics of quantum neural networks.

REFERENCES

- [1] Enrique Barba Roque and Luis Cruz. “Energy Aware Development of Neuromorphic Implantables: From Metrics to Action”. In: *arXiv e-prints* (2025), arXiv–2506.
- [2] John Preskill. “Quantum computing in the NISQ era and beyond”. In: *Quantum* 2 (2018), p. 79.
- [3] Nicola Peserico, Bhavin J Shastri, and Volker J Sorger. “Integrated photonic tensor processing unit for a matrix multiply: a review”. In: *Journal of Lightwave Technology* 41.12 (2023), pp. 3704–3716.
- [4] AV Krishnamoorthy et al. “Energy-efficient photonics in future high-connectivity computing systems”. In: *Journal Of Lightwave Technology* 33.4 (2015), pp. 889–900.
- [5] Jacques Carolan et al. “Universal linear optics”. In: *Science* 349.6249 (2015), pp. 711–716.
- [6] Stefan Scheel et al. “Measurement-induced nonlinearity in linear optics”. In: *Physical Review A* 68.3 (2003), p. 032310.
- [7] Alexia Salavrakos et al. “Photon-native quantum algorithms”. In: *Materials for Quantum Technology* 5.2 (2025), p. 023001.
- [8] Michael Reck et al. “Experimental realization of any discrete unitary operator”. In: *Physical Review Letters* 73.1 (1994), pp. 58–61. DOI: [10.1103/PhysRevLett.73.58](https://doi.org/10.1103/PhysRevLett.73.58).
- [9] William R. Clements et al. “Optimal design for universal multiport interferometers”. In: *Optica* 3.12 (2016), pp. 1460–1465. DOI: [10.1364/OPTICA.3.001460](https://doi.org/10.1364/OPTICA.3.001460).
- [10] Raúl García-Patrón, Jelmer J Renema, and Valery Shchesnovich. “Simulating boson sampling in lossy architectures”. In: *Quantum* 3 (2019), p. 169.
- [11] Scott Aaronson and Alex Arkhipov. “The Computational Complexity of Linear Optics”. In: *Theory of Computing* 9.4 (2013), pp. 143–252. DOI: [10.4086/toc.2013.v009a004](https://doi.org/10.4086/toc.2013.v009a004).
- [12] Kamil Brádler et al. “Gaussian boson sampling for perfect matchings of arbitrary graphs”. In: *Physical Review A* 98.3 (2018), p. 032310.
- [13] Tigran Sedrakyan and Alexia Salavrakos. “Photonic quantum generative adversarial networks for classical data”. In: *Optica Quantum* 2.6 (2024), pp. 458–467.
- [14] Zhenghao Yin et al. “Experimental quantum-enhanced kernels on a photonic processor”. In: *arXiv preprint arXiv:2407.20364* (2024).
- [15] Beng Yee Gan, Daniel Leykam, and Dimitris G Angelakis. “Fock state-enhanced expressivity of quantum machine learning models”. In: *EPJ Quantum Technology* 9.1 (2022), p. 16.
- [16] Léo Monbroussou et al. “Toward quantum advantage with photonic state injection”. In: *Physical Review Research* 7.3 (2025), p. 033051.
- [17] Ulysse Chabaud, Damian Markham, and Adel Sohbi. “Quantum machine learning with adaptive linear optics”. In: *Quantum* 5 (2021), p. 496.
- [18] Wulfram Gerstner et al. *Neuronal dynamics: From single neurons to networks and models of cognition*. <https://neurondynamics.epfl.ch/>. Cambridge University Press, 2014.
- [19] Warren S McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [20] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.
- [21] Marvin Minsky and Seymour A Papert. *Perceptrons, reissue of the 1988 expanded edition with a new foreword by Léon Bottou: an introduction to computational geometry*. MIT press, 2017.
- [22] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. “Activation functions in deep learning: A comprehensive survey and benchmark”. In: *Neurocomputing* 503 (2022), pp. 92–108.

- [23] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural networks* 2.5 (1989), pp. 359–366.
- [24] Moshe Leshno et al. “Multilayer feedforward networks with a nonpolynomial activation function can approximate any function”. In: *Neural networks* 6.6 (1993), pp. 861–867.
- [25] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the difficulty of training recurrent neural networks”. In: *International conference on machine learning*. PMLR. 2013, pp. 1310–1318.
- [26] Kaifeng Lyu and Jian Li. “Gradient Descent Maximizes the Margin of Homogeneous Neural Networks”. In: *International Conference on Learning Representations*. 2020.
- [27] Mihaela Rosca et al. “A case for new neural network smoothness constraints”. In: *Proceedings of Machine Learning Research* (2020).
- [28] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [29] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.
- [30] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *nature* 521.7553 (2015), pp. 436–444.
- [31] Wolfgang Maass. “Networks of spiking neurons: the third generation of neural network models”. In: *Neural networks* 10.9 (1997), pp. 1659–1671.
- [32] Wulfram Gerstner and Werner M Kistler. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.
- [33] Wulfram Gerstner. “The spike response model”. In: *The Handbook of Biological Physics* 4 (1999), pp. 469–516.
- [34] Renaud Jolivet, Timothy J Lewis, and Wulfram Gerstner. “Generalized integrate-and-fire models of neuronal activity approximate spike trains of a detailed model to a high degree of accuracy”. In: *Journal of neurophysiology* 92.2 (2004), pp. 959–976.
- [35] Manjot Singh, Adalbert Fono, and Gitta Kutyniok. “Expressivity of Spiking Neural Networks through the Spike Response Model”. In: *UniReps: the First Workshop on Unifying Representations in Neural Models*. 2023.
- [36] Gabriele Lagani et al. “Spiking neural networks and bio-inspired supervised deep learning: a survey”. In: *arXiv preprint arXiv:2307.16235* (2023).
- [37] Maurizio Mattia and Paolo Del Giudice. “Population dynamics of interacting spiking neurons”. In: *Physical Review E* 66.5 (2002), p. 051917.
- [38] Wulfram Gerstner. “Population dynamics of spiking neurons: fast transients, asynchronous states, and locking”. In: *Neural computation* 12.1 (2000), pp. 43–89.
- [39] David Liu and Máté Lengyel. “Bayesian nonparametric (non-) renewal processes for analyzing neural spike train variability”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 68013–68027.
- [40] Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. “Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks”. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63.
- [41] Yaoyu Zhu et al. “Training spiking neural networks with event-driven backpropagation”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 30528–30541.
- [42] Richard Kempster, Wulfram Gerstner, and J van Hemmen. “Spike-based compared to rate-based Hebbian learning”. In: *Advances in neural information processing systems* 11 (1998).
- [43] Michael Beyeler, Nikil D Dutt, and Jeffrey L Krichmar. “Categorization and decision-making in a neurobiologically plausible spiking network using a STDP-like learning rule”. In: *Neural Networks* 48 (2013), pp. 109–124.

- [44] Timo C Wunderlich and Christian Pehle. “Event-based backpropagation can compute exact gradients for spiking neural networks”. In: *Scientific Reports* 11.1 (2021), p. 12829.
- [45] H  lene Paugam-Moisy. *Spiking neuron networks a survey*. Tech. rep. IDIAP, 2006.
- [46] Wolfgang Maass. “On the computational power of noisy spiking neurons”. In: *Advances in neural information processing systems* 8 (1995).
- [47] Marc de Kamps and Frank van der Velde. “Implementation of multilayer perceptron networks by populations of spiking neurons using rate coding”. In: *Neurocomputing* 44 (2002), pp. 353–358.
- [48] Wolfgang Maass. “Fast sigmoidal networks via spiking neurons”. In: *Neural computation* 9.2 (1997), pp. 279–304.
- [49] Marc de Kamps and Frank van der Velde. “From artificial neural networks to spiking neuron populations and back again”. In: *Neural Networks* 14.6-7 (2001), pp. 941–953.
- [50] Tong Bu et al. “Optimal ANN-SNN Conversion for High-accuracy and Ultra-low-latency Spiking Neural Networks”. In: *International Conference on Learning Representations*. 2023.
- [51] Yuchen Wang et al. “A universal ANN-to-SNN framework for achieving high accuracy and low latency deep Spiking Neural Networks”. In: *Neural Networks* 174 (2024), p. 106244.
- [52] Loc Vu-Quoc. *Neuron and myelinated axon (Neuron3.png)*. Wikimedia Commons. CC BY-SA 4.0. Accessed 2025-08-31. Sept. 2018. URL: <https://commons.wikimedia.org/w/index.php?curid=72816083>.
- [53] Marcus Stoffel and Saurabh Balkrishna Tandale. “Spiking neural networks for nonlinear regression of complex transient signals on sustainable neuromorphic processors”. In: *npj Unconventional Computing* 1.1 (2024), p. 2.
- [54] Allan Pinkus. “Approximation theory of the MLP model in neural networks”. In: *Acta numerica* 8 (1999), pp. 143–195.
- [55] Marius-Constantin Popescu et al. “Multilayer perceptron and neural networks”. In: *WSEAS Transactions on Circuits and Systems* 8.7 (2009), pp. 579–588.
- [56] Gal Vardi and Ohad Shamir. “Neural networks with small weights and depth-separation barriers”. In: *Advances in neural information processing systems* 33 (2020), pp. 19433–19442.
- [57] Muhammad Shafiq and Zhaoquan Gu. “Deep residual learning for image recognition: A survey”. In: *Applied sciences* 12.18 (2022), p. 8972.
- [58] Li Shen et al. “On efficient training of large-scale deep learning models”. In: *ACM Computing Surveys* 57.3 (2024), pp. 1–36.
- [59] Maria Schuld and Nathan Killoran. “Quantum machine learning in feature Hilbert spaces”. In: *Physical review letters* 122.4 (2019), p. 040504.
- [60] Marco Cerezo et al. “Challenges and opportunities in quantum machine learning”. In: *Nature computational science* 2.9 (2022), pp. 567–576.
- [61] Jonas K  bler, Simon Buchholz, and Bernhard Sch  lkopf. “The inductive bias of quantum kernels”. In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 12661–12673.
- [62] Jarrod R McClean et al. “Barren plateaus in quantum neural network training landscapes”. In: *Nature communications* 9.1 (2018), p. 4812.
- [63] Edward Grant et al. “An initialization strategy for addressing barren plateaus in parametrized quantum circuits”. In: *Quantum* 3 (2019), p. 214.
- [64] Marcello Benedetti et al. “Parameterized quantum circuits as machine learning models”. In: *Quantum science and technology* 4.4 (2019), p. 043001.
- [65] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [66] John Watrous. *The theory of quantum information*. Cambridge university press, 2018.

- [67] Mark M Wilde. *Quantum information theory*. Cambridge university press, 2013.
- [68] John Preskill. *Lecture Notes on Quantum Information and Quantum Computation*. Caltech Lecture Notes. 2018.
- [69] Subhash C Kak. “Quantum neural computing”. In: *Advances in imaging and electron physics* 94 (1995), pp. 259–313.
- [70] Ronald Chrisley. “Quantum learning”. In: *New directions in cognitive science: Proceedings of the international symposium, Saariselka*. Vol. 4. 1995.
- [71] M Lewenstein. “Quantum Perceptrons”. In: *Journal of Modern Optics* 41.12 (1994), pp. 2491–2501.
- [72] Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. “The quest for a quantum neural network”. In: *Quantum Information Processing* 13.11 (2014), pp. 2567–2586.
- [73] Maria Schuld and Nathan Killoran. “Is quantum advantage the right goal for quantum machine learning?” In: *Prx Quantum* 3.3 (2022), p. 030101.
- [74] Kerstin Beer et al. “Training deep quantum neural networks”. In: *Nature communications* 11.1 (2020), p. 808.
- [75] Francesco Tacchino et al. “An artificial neuron implemented on an actual quantum processor”. In: *npj Quantum Information* 5.1 (2019), pp. 1–8.
- [76] Stefano Mangini et al. “Quantum computing model of an artificial neuron with continuously valued input data”. In: *Machine Learning: Science and Technology* 1.4 (2020), p. 045008.
- [77] Alexandre Y Yamamoto et al. “Simulation of a multidimensional input quantum perceptron”. In: *Quantum Information Processing* 17.6 (2018), p. 128.
- [78] Cláudio A Monteiro et al. “Quantum neuron with real weights”. In: *Neural Networks* 143 (2021), pp. 698–708.
- [79] JJ Torres and D Manzano. “A model of interacting quantum neurons with a dynamic synapse”. In: *New Journal of Physics* 24.7 (2022), p. 073007.
- [80] Fernando M de Paula Neto et al. “Chaos in a quantum neuron: An open system approach”. In: *Neurocomputing* 246 (2017), pp. 3–11.
- [81] Changpeng Shao. “A quantum model of feed-forward neural networks with unitary learning algorithms”. In: *Quantum Information Processing* 19.3 (2020), p. 102.
- [82] Fernando M de Paula Neto et al. “Quantum perceptron with dynamic internal memory”. In: *2018 International Joint Conference on Neural Networks (IJCNN)*. IEEE. 2018, pp. 1–8.
- [83] Ashish Kapoor, Nathan Wiebe, and Krysta Svore. “Quantum perceptron models”. In: *Advances in neural information processing systems* 29 (2016).
- [84] Adenilton José da Silva, Teresa Bernarda Ludermir, and Wilson Rosa de Oliveira. “Quantum perceptron over a field and neural network architecture selection in a quantum computer”. In: *Neural Networks* 76 (2016), pp. 55–64.
- [85] T Espinosa-Ortega and TCH Liew. “Perceptrons with Hebbian learning based on wave ensembles in spatially patterned potentials”. In: *Physical Review Letters* 114.11 (2015), p. 118101.
- [86] Claudia Artiaco et al. “Quantum jamming: Critical properties of a quantum mechanical perceptron”. In: *Physical Review A* 103.4 (2021), p. L040203.
- [87] Jonathan HA de Carvalho and Fernando M de Paula Neto. “Parametrized constant-depth quantum neuron”. In: *IEEE Transactions on Neural Networks and Learning Systems* 35.11 (2023), pp. 15932–15943.
- [88] Shilu Yan, Hongsheng Qi, and Wei Cui. “Nonlinear quantum neuron: A fundamental building block for quantum neural networks”. In: *Physical Review A* 102.5 (2020), p. 052421.
- [89] Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. “Simulating a perceptron on a quantum computer”. In: *Physics Letters A* 379.7 (2015), pp. 660–663.

- [90] Yudong Cao, Gian Giacomo Guerreschi, and Alán Aspuru-Guzik. “Quantum neuron: an elementary building block for machine learning on quantum computers”. In: *arXiv preprint arXiv:1711.11240* (2017).
- [91] Panchi Li and Bing Wang. “Quantum neural networks model based on swap test and phase estimation”. In: *Neural Networks* 130 (2020), pp. 152–164.
- [92] Kunal Sharma et al. “Trainability of dissipative perceptron-based quantum neural networks”. In: *Physical Review Letters* 128.18 (2022), p. 180505.
- [93] Deniz Türkpençe. “Reservoir induced activation of a quantum neuron”. In: *Physics Letters A* 384.23 (2020), p. 126442.
- [94] Ufuk Korkmaz and Deniz Türkpençe. “Quantum collisional classifier driven by information reservoirs”. In: *Physical Review A* 107.1 (2023), p. 012432.
- [95] Roberto Bondesan and Max Welling. “Quantum deformed neural networks”. In: *arXiv preprint arXiv:2010.11189* (2020).
- [96] Fabio Benatti, Stefano Mancini, and Stefano Mangini. “Continuous variable quantum perceptron”. In: *International Journal of Quantum Information* 17.08 (2019), p. 1941009.
- [97] Erik Torrontegui and Juan José García-Ripoll. “Unitary quantum perceptron as efficient universal approximator (a)”. In: *Europhysics Letters* 125.3 (2019), p. 30004.
- [98] Yue Ban et al. “Speeding up quantum perceptron via shortcuts to adiabaticity”. In: *Scientific reports* 11.1 (2021), p. 5783.
- [99] Seth Lloyd et al. “Quantum embeddings for machine learning”. In: *arXiv preprint arXiv:2001.03622* (2020).
- [100] Martin Larocca et al. “Barren plateaus in variational quantum computing”. In: *Nature Reviews Physics* (2025), pp. 1–16.
- [101] Antonio Sannia et al. “Engineered dissipation to mitigate barren plateaus”. In: *npj Quantum Information* 10.1 (2024), p. 81.
- [102] Joseph Bowles et al. “Contextuality and inductive bias in quantum machine learning”. In: *arXiv preprint arXiv:2302.01365* (2023).
- [103] Kaitlin Gili, Guillermo Alonso, and Maria Schuld. “An inductive bias from quantum mechanics: learning order effects with non-commuting measurements”. In: *Quantum Machine Intelligence* 6.2 (2024), p. 67.
- [104] Roeland Wiersema and HJ Kappen. “Implementing perceptron models with qubits”. In: *Physical Review A* 100.2 (2019), p. 020301.
- [105] Marco Maronese, Claudio Destri, and Enrico Prati. “Quantum activation functions for quantum neural networks”. In: *Quantum Information Processing* 21.4 (2022), p. 128.
- [106] P Huber et al. “Realization of a quantum perceptron gate with trapped ions”. In: *arXiv preprint arXiv:2111.08977* (2021).
- [107] Ishita Agarwal et al. “Extending quantum perceptrons: Rydberg devices, multi-class classification, and error tolerance”. In: *arXiv preprint arXiv:2411.09093* (2024).
- [108] Lasse Bjørn Kristensen et al. “An artificial spiking quantum neuron”. In: *npj Quantum Information* 7.1 (2021), p. 59.
- [109] Dean Brand and Francesco Petruccione. “A quantum leaky integrate-and-fire spiking neuron and network”. In: *npj Quantum Information* 10.1 (2024), p. 125.
- [110] Tasio Gonzalez Raya, Mikel Sanz, and Enrique Solano. “Quantized Hodgkin-Huxley Model for Quantum Neurons”. In: *Bulletin of the American Physical Society* 65 (2020).
- [111] Yinqian Sun, Yi Zeng, and Tielin Zhang. “Quantum superposition inspired spiking neural network”. In: *Iscience* 24.8 (2021).

- [112] James C Spall. “Multivariate stochastic approximation using a simultaneous perturbation gradient approximation”. In: *IEEE transactions on automatic control* 37.3 (2002), pp. 332–341.
- [113] James C Spall. “Implementation of the simultaneous perturbation algorithm for stochastic optimization”. In: *IEEE Transactions on aerospace and electronic systems* 34.3 (2002), pp. 817–823.
- [114] Nicolas Maring et al. “A versatile single-photon-based quantum computing platform”. In: *Nature Photonics* 18.6 (2024), pp. 603–609.
- [115] Quandela. *Belenos*. <https://www.quandela.com/qpus/belenos/>. Accessed: 2025-08-31.
- [116] Nicolas Heurtel et al. “Perceval: A software platform for discrete variable photonic quantum computing”. In: *Quantum* 7 (2023), p. 931.